

PROPOSAL CAPSTONE PROJECT

DETEKSI DAFTAR PUSTAKA HALU

Kelompok 7 :

1. M. Rayhan Saputra (11231061)
2. Daniel Belawa Koten
(11231019)
3. M. Rivaldi (11231062)
4. Fadhil Zulfikar (11231024)
5. Bagus Nur. A (11231015)
6. Juan Resq Sukardi (11231033)
7. Andy Cole (11221079)

**Program Studi Informatika
Jurusan Teknologi Informasi dan Bisnis
Fakultas Sains dan Teknologi Informasi
Institut Teknologi Kalimantan**

2026

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR GAMBAR.....	5
DAFTAR TABEL.....	7
BAB 1.....	8
PENDAHULUAN.....	8
1.1 Latar Belakang.....	8
1.2 Rumusan Masalah.....	9
1.3 Tujuan Proyek.....	9
1.4 Batasan Masalah.....	10
BAB 2.....	11
TINJAUAN PUSTAKA.....	11
2.1 Daftar Pustaka.....	11
2.2 Sitasi.....	11
2.3 Digital Object Identifier (DOI).....	12
2.4 Crossref API.....	12
2.5 GROBID.....	13
2.6 Fuzzy Matching.....	14
2.6.1. Levenshtein Distance.....	14
2.6.2. Jaro-Winkler.....	15
2.7 Sentence-BERT (SBERT).....	16
2.8 Evaluasi Sistem Deteksi.....	17
2.8.1. Confusion Matrix.....	17
2.8.2. Precision.....	18
2.8.3. Recall.....	18
2.8.4. F1-Score.....	19
2.9 Agile.....	19
Gambar 2.9 Tahapan Agile.....	20
BAB 3.....	21
METODOLOGI.....	21
3.1 Diagram Alir Proyek.....	21
3.1.1. Diagram Alir Pembuatan Sistem.....	21
Gambar 3.1.1 Diagram Alir Pembuatan Sistem.....	21
3.1.2. Diagram Alir Pembuatan Model AI Deteksi.....	22
Gambar 3.1.2 Diagram Alir Pembuatan Model AI Deteksi.....	23
3.2 Use Case Diagram.....	24
3.2.1. Use Case Authentication.....	24
Gambar 3.2.1. Use Case Authion.....	24
3.2.2. Use Case System.....	25
Gambar 3.2.2. Use Case System.....	25
3.3 Activity Diagram.....	27
3.3.1. Activity Register.....	27
Gambar 3.3.1. Activity Register.....	27

3.3.2. Activity Login.....	28
Gambar 3.3.2. Activity Login.....	28
3.3.3. Activity Logout.....	29
Gambar 3.3.3. Activity Logout.....	29
3.3.4. Activity Mengunggah Dokumen.....	30
Gambar 3.3.4. Activity Mengunggah Dokumen.....	30
3.3.5. Activity Mengecek Dokumen.....	32
3.3.6. Activity Mendeteksi Daftar Pustaka.....	34
3.3.7. Activity Mendeteksi Sitasi.....	36
3.3.8. Activity Melihat Hasil Pengecekan Dokumen.....	37
Gambar 3.3.8. Activity Melihat Hasil Pengecekan Dokumen.....	37
3.3.9. Activity Melihat Riwayat Dokumen.....	38
Gambar 3.3.9. Activity Melihat Riwayat Dokumen.....	38
3.3.10. Activity Menghapus Riwayat Dokumen.....	39
Gambar 3.3.10. Activity Menghapus Riwayat Dokumen.....	39
3.3.11. Activity Mengunduh Hasil Dokumen yang di-cek.....	40
Gambar 3.3.11. Activity Mengunduh Hasil Dokumen yang di-cek.....	40
3.3.12. Activity Merevisi Hasil Pengecekan Dokumen.....	41
Gambar 3.3.12. Activity Merevisi Hasil Pengecekan Dokumen.....	41
3.4 Sequence Diagram.....	43
3.4.1. Register.....	43
Gambar 3.3.1. Sequence Register.....	43
3.4.2. Sequence Login.....	44
Gambar 3.3.2. Sequence Login.....	44
3.4.3. Sequence Logout.....	45
Gambar 3.3.3. Sequence Logout.....	45
3.4.4. Sequence Mengunggah Dokumen.....	46
Gambar 3.3.4. Sequence Mengunggah Dokumen.....	46
3.4.5. Sequence Mengecek Dokumen.....	47
Gambar 3.4.5. Sequence Mengecek Dokumen.....	47
3.4.6. Sequence Mendeteksi Daftar Pustaka.....	49
Gambar 3.4.6. Sequence Mendeteksi Daftar Pustaka.....	49
3.4.7. Sequence Mendeteksi Sitasi.....	50
Gambar 3.4.7. Sequence Mendeteksi Sitasi.....	50
3.4.8. Sequence Melihat Hasil Pengecekan Dokumen.....	51
Gambar 3.4.8. Sequence Melihat Hasil Pengecekan Dokumen.....	51
3.4.9. Sequence Melihat Riwayat Dokumen.....	52
Gambar 3.4.9. Sequence Melihat Riwayat Dokumen.....	52
3.4.10. Sequence Menghapus Riwayat Dokumen.....	53
Gambar 3.4.10. Sequence Menghapus Riwayat Dokumen.....	53
3.4.11. Sequence Mengunduh Hasil Dokumen yang di-cek.....	54
Gambar 3.4.11. Sequence Mengunduh Hasil Dokumen yang di-cek.....	54
3.4.12. Sequence Merevisi Hasil Pengecekan Dokumen.....	55
3.5 Class Diagram.....	58

3.5.1. Class Diagram Authentication.....	58
Gambar 3.5.1. Class Diagram Authentication.....	59
3.5.2. Class Diagram Upload Dokumen.....	59
Gambar 3.5.2. Class Diagram Upload Dokumen.....	60
3.5.3. Class Diagram Mengecek Dokumen.....	60
Gambar 3.5.3. Class Diagram Mengecek Dokumen.....	61
3.5.4. Class Diagram Melihat Hasil, Riwayat, dan Mengunduh Laporan.....	61
Gambar 3.5.4. Class Diagram Melihat Hasil, Riwayat, dan Mengunduh Laporan..	62
3.5.5. Class Diagram Revisi Hasil Pengecekan.....	63
Gambar 3.5.5. Class Diagram Revisi Hasil Pengecekan.....	63
3.6 Entity Relationship Diagram.....	64
3.7 Desain Antarmuka.....	65
3.7.1. Desain Antarmuka Halaman Unggah Dokumen.....	65
Gambar 3.7.1. Desain Antarmuka Halaman Unggah Dokumen.....	65
3.7.2. Desain Antarmuka Setelah Dokumen Dipilih.....	65
Gambar 3.7.2. Desain Antarmuka Setelah Dokumen Dipilih.....	65
3.7.3. Desain Antarmuka Proses Analisis.....	66
Gambar 3.7.3. Desain Antarmuka Proses Analisis.....	66
3.7.4. Desain Antarmuka Hasil Pengecekan.....	67
Gambar 3.7.4. Desain Antarmuka Hasil Pengecekan.....	67
3.7.5. Desain Antarmuka Detail Status Referensi.....	68
Gambar 3.7.5. Desain Antarmuka Detail Status Referensi.....	68
3.7.6. Desain Antarmuka Riwayat Analisis.....	69
Gambar 3.7.6. Desain Antarmuka Riwayat Analisis.....	69
BAB 4.....	70
RENCANA PELAKSANAAN.....	70
4.1 Rencana Kegiatan.....	70
Tabel 4.1 Rencana Kegiatan.....	70
4.2 Pembagian Tugas.....	71
Tabel 4.2 Pembagian Tugas.....	71
DAFTAR PUSTAKA.....	73

DAFTAR GAMBAR

Gambar 2.9 Tahapan Agile.....	20
Gambar 3.1.1 Diagram Alir Pembuatan Sistem.....	21
Gambar 3.1.2 Diagram Alir Pembuatan Model AI Deteksi.....	23
Gambar 3.2.1. Use Case Authion.....	24
Gambar 3.2.2. Use Case System.....	25
Gambar 3.3.1. Activity Register.....	27
Gambar 3.3.2. Activity Login.....	28
Gambar 3.3.3. Activity Logout.....	29
Gambar 3.3.4. Activity Mengunggah Dokumen.....	30
Gambar 3.3.8. Activity Melihat Hasil Pengecekan Dokumen.....	37
Gambar 3.3.9. Activity Melihat Riwayat Dokumen.....	38
Gambar 3.3.10. Activity Menghapus Riwayat Dokumen.....	39
Gambar 3.3.11. Activity Mengunduh Hasil Dokumen yang di-cek.....	40
Gambar 3.3.12. Activity Merevisi Hasil Pengecekan Dokumen.....	41
Gambar 3.3.1. Sequence Register.....	43
Gambar 3.3.2. Sequence Login.....	44
Gambar 3.3.3. Sequence Logout.....	45
Gambar 3.3.4. Sequence Mengunggah Dokumen.....	46
Gambar 3.4.5. Sequence Mengecek Dokumen.....	47
Gambar 3.4.6. Sequence Mendeteksi Daftar Pustaka.....	49
Gambar 3.4.7. Sequence Mendeteksi Sitasi.....	50
Gambar 3.4.8. Sequence Melihat Hasil Pengecekan Dokumen.....	51
Gambar 3.4.9. Sequence Melihat Riwayat Dokumen.....	52
Gambar 3.4.10. Sequence Menghapus Riwayat Dokumen.....	53
Gambar 3.4.11. Sequence Mengunduh Hasil Dokumen yang di-cek.....	54
Gambar 3.5.1. Class Diagram Authentication.....	59
Gambar 3.5.2. Class Diagram Upload Dokumen.....	60
Gambar 3.5.3. Class Diagram Mengecek Dokumen.....	61
Gambar 3.5.4. Class Diagram Melihat Hasil, Riwayat, dan Mengunduh Laporan.....	62
Gambar 3.5.5. Class Diagram Revisi Hasil Pengecekan.....	63
Gambar 3.7.1. Desain Antarmuka Halaman Unggah Dokumen.....	65
Gambar 3.7.2. Desain Antarmuka Setelah Dokumen Dipilih.....	65
Gambar 3.7.3. Desain Antarmuka Proses Analisis.....	66
Gambar 3.7.4. Desain Antarmuka Hasil Pengecekan.....	67
Gambar 3.7.5. Desain Antarmuka Detail Status Referensi.....	68
Gambar 3.7.6. Desain Antarmuka Riwayat Analisis.....	69

DAFTAR TABEL

Tabel 4.1 Rencana Kegiatan.....	69
Tabel 4.2 Pembagian Tugas.....	70

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Penggunaan *generative AI* seperti *ChatGPT* dalam penyusunan karya tulis ilmiah, termasuk Tugas Akhir mahasiswa, telah berkembang pesat dalam beberapa tahun terakhir. Namun, di balik kemudahan yang ditawarkan, muncul persoalan serius berupa referensi fiktif atau tidak akurat yang dikenal dengan istilah "*hallucinated citations*", "*ghost references*", atau "*phantom references*". Walters dan Wilder (2023) menemukan bahwa ketika *ChatGPT-3.5* dan *ChatGPT-4* diminta menghasilkan tinjauan literatur berisi 636 sitasi, sebanyak 55% sitasi dari *ChatGPT-3.5* dan 18% sitasi dari *ChatGPT-4* sepenuhnya fiktif, dengan pola referensi palsu yang sering menggabungkan nama penulis asli dengan judul karangan. Yang lebih mengkhawatirkan, Boretti (2026) melaporkan bahwa audit terhadap 4.841 makalah yang diterima di NeurIPS 2025, salah satu konferensi AI paling bergengsi, menemukan setidaknya 100 sitasi halu yang tersebar di 53 makalah meskipun telah melalui proses tinjauan sejawat yang ketat oleh 3–5 pakar per makalah. Fakta bahwa publikasi ilmiah dengan proses seleksi seketat itu masih kebobolan menunjukkan bahwa mahasiswa yang menyusun Tugas Akhir tanpa lapisan verifikasi seketat itu jauh lebih rentan terhadap masalah yang sama, sehingga dibutuhkan mekanisme deteksi otomatis untuk menjaga integritas akademik.

Untuk menjawab permasalahan tersebut, penelitian ini mengusulkan sebuah aplikasi web yang mampu memvalidasi eksistensi referensi secara otomatis sekaligus mencocokkan konsistensi sitasi di dalam dokumen Tugas Akhir. Sistem ini dirancang untuk mendeteksi tiga kondisi utama: pertama, referensi dengan DOI yang tercantum namun tidak valid atau mengarah ke publikasi yang berbeda dari yang dituliskan; kedua, referensi yang sudah sesuai kaidah format penulisan namun belum mencantumkan DOI, di mana sistem akan mencari DOI yang paling sesuai; dan ketiga, sitasi dalam naskah yang tidak memiliki pasangan sah di daftar pustaka atau memiliki kombinasi nama-tahun yang keliru. Hasil deteksi ditampilkan secara visual melalui penandaan warna pada teks, sehingga pengguna dapat langsung menelusuri bagian yang bermasalah tanpa perlu membaca ulang keseluruhan dokumen secara manual. Selain itu, sistem turut menyimpan riwayat setiap dokumen yang telah diperiksa agar hasil pengecekan sebelumnya dapat diakses kembali sewaktu-waktu, menyerupai mekanisme yang lazim digunakan pada perangkat lunak deteksi plagiarisme di lingkungan akademik.

Guna mewujudkan solusi tersebut, sistem ini memanfaatkan beberapa teknologi pendukung yang efektivitasnya telah dibuktikan oleh penelitian-penelitian terkini. Untuk ekstraksi struktur bibliografi dari dokumen yang tidak terstandar, digunakan pustaka pengurai referensi seperti GROBID, yang menurut studi *benchmark* oleh Meuschke dkk. (2023) terhadap sepuluh alat ekstraksi PDF, terbukti memiliki performa terbaik dalam tugas ekstraksi referensi bibliografi dibandingkan alat

sejenis lainnya. Validasi eksistensi referensi memanfaatkan *Crossref API* yang dikombinasikan dengan *fuzzy text matching*, pendekatan yang juga digunakan Ilter (2026) dalam audit forensik terhadap 5.514 sitasi pada 50 makalah survei bidang kecerdasan buatan untuk membedakan sitasi yang benar-benar tidak ada (phantom) dari sekadar kesalahan format penulisan. Untuk mengukur kemiripan makna antar judul referensi digunakan model embedding kalimat pretrained seperti Sentence-BERT (SBERT), pendekatan yang sejalan dengan sistem *SemanticCite* yang dikembangkan Haan (2025) untuk verifikasi sitasi berbasis analisis semantik teks secara otomatis. Pencocokan nama penulis dan tahun publikasi pada sitasi in-text turut memanfaatkan algoritma *fuzzy string matching* seperti *Levenshtein Distance* dan *Jaro-Winkler*.

Masing-masing teknologi tersebut berperan saling melengkapi dalam menjawab solusi yang ditawarkan. *Crossref API* berfungsi sebagai sumber kebenaran eksternal yang independen dari dokumen itu sendiri, sehingga validasi tidak hanya bergantung pada asumsi bahwa referensi yang tampak rapi pasti benar-benar ada. *SBERT* berperan menjembatani perbedaan redaksional antara judul yang tertulis di dokumen dengan judul asli pada metadata publikasi, sehingga sistem tetap dapat mendeteksi kecocokan meskipun tidak identik kata per kata. Algoritma *fuzzy matching* seperti *Levenshtein Distance* dan *Jaro-Winkler* memungkinkan sistem mencocokkan sitasi in-text terhadap daftar pustaka meskipun terdapat variasi kecil dalam penulisan nama penulis. Kombinasi dari seluruh teknologi ini memungkinkan proses deteksi referensi halu berjalan secara otomatis, konsisten, dan lebih efisien dibandingkan pengecekan manual yang lambat, rawan kesalahan manusia, serta sulit dilakukan secara menyeluruh pada dokumen dengan jumlah referensi yang banyak.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Seberapa akurat sistem dalam mendeteksi referensi dengan DOI yang tidak valid atau halu pada daftar pustaka dokumen?
2. Seberapa akurat sistem dalam menemukan dan menyarankan DOI yang sesuai untuk referensi yang telah sesuai format penulisan namun belum mencantumkan DOI?
3. Seberapa akurat sistem dalam mendeteksi sitasi in-text yang tidak memiliki pasangan sah di daftar pustaka atau memiliki kombinasi nama-tahun yang tidak sesuai?

1.3 Tujuan Proyek

Berdasarkan rumusan masalah yang telah diuraikan, tujuan penelitian ini adalah sebagai berikut:

1. Mengetahui tingkat akurasi sistem dalam mendeteksi referensi dengan DOI yang tidak valid atau halu pada daftar pustaka dokumen Tugas Akhir, yang diukur

menggunakan Precision, Recall, F1-Score, dan Confusion Matrix terhadap dataset ground truth yang telah dilabeli.

2. Mengetahui tingkat akurasi sistem dalam menemukan dan menyarankan DOI yang sesuai untuk referensi yang telah sesuai format penulisan namun belum mencantumkan DOI, yang diukur menggunakan Precision dan Recall terhadap kecocokan DOI hasil pencarian sistem dengan DOI sebenarnya pada dataset ground truth.
3. Mengetahui tingkat akurasi sistem dalam mendeteksi sitasi in-text yang tidak memiliki pasangan sah di daftar pustaka atau memiliki kombinasi nama-tahun yang tidak sesuai, yang diukur menggunakan Precision, Recall, F1-Score, dan Confusion Matrix terhadap dataset ground truth yang telah dilabeli.

1.4 Batasan Masalah

Agar penelitian ini lebih terarah dan sesuai dengan waktu serta sumber daya yang tersedia, ditetapkan batasan-batasan berikut:

1. Dokumen yang dapat diproses oleh sistem terbatas pada format PDF dan DOCX, sehingga format lain seperti gambar hasil pindai (*scan*) tanpa lapisan teks tidak didukung.
2. Gaya sitasi yang dapat divalidasi formatnya terbatas pada APA dan IEEE, karena keduanya merupakan gaya yang paling umum digunakan pada Tugas Akhir di lingkungan akademik.
3. Validasi eksistensi referensi hanya mengandalkan basis data eksternal *Crossref API* dan *OpenAlex API*, sehingga referensi yang tidak terindeks di kedua basis data tersebut seperti beberapa jurnal atau prosiding lokal berbahasa Indonesia berpotensi tidak dapat divalidasi secara akurat dan akan ditandai sebagai "tidak ditemukan", bukan berarti pasti halu.
4. Sistem tidak menggunakan *Google Scholar* sebagai sumber validasi karena tidak tersedianya API resmi dan adanya batasan penggunaan (*Terms of Service*) yang melarang scraping.
5. Sistem berfokus pada validasi DOI dan konsistensi sitasi (tiga kasus: DOI halu, DOI hilang pada referensi valid, dan sitasi in-text yang tidak sesuai daftar pustaka), dan tidak mencakup deteksi plagiarisme konten, pengecekan tata bahasa, maupun penilaian kualitas substansi tulisan.
6. Proses validasi bergantung pada ketersediaan dan batas kuota (*rate limit*) API eksternal, sehingga kecepatan dan hasil pemrosesan dapat dipengaruhi oleh kondisi layanan pihak ketiga tersebut di luar kendali sistem.

BAB 2

TINJAUAN PUSTAKA

2.1 Daftar Pustaka

Daftar pustaka merupakan kumpulan sumber yang digunakan dan dirujuk dalam suatu karya ilmiah. Daftar pustaka berfungsi memberikan informasi yang memungkinkan pembaca mengidentifikasi dan menemukan kembali sumber yang digunakan oleh penulis. Dalam *Publication Manual of the American Psychological Association* edisi ke-7, daftar referensi ditempatkan pada bagian akhir karya ilmiah dan setiap entri memuat informasi bibliografis yang diperlukan untuk mengidentifikasi sumber, seperti penulis, tahun, judul, dan sumber publikasi (American Psychological Association, 2020). Penyusunan daftar pustaka juga perlu dilakukan secara konsisten sesuai dengan gaya sitasi yang digunakan dalam suatu karya ilmiah.

Daftar pustaka memiliki keterkaitan langsung dengan sumber yang dikutip di dalam teks. Dalam penulisan akademik, sumber yang dicantumkan dalam daftar referensi harus dapat ditelusuri dan sesuai dengan sumber yang dirujuk dalam teks. APA Style memberikan aturan khusus mengenai elemen referensi, variasi berdasarkan jenis sumber, serta pengurutan dan format daftar referensi (American Psychological Association, 2020). Dengan demikian, pemeriksaan daftar pustaka tidak hanya berkaitan dengan kesesuaian format, tetapi juga dengan keberadaan dan ketepatan informasi bibliografis yang digunakan.

Dalam penelitian ini, daftar pustaka menjadi salah satu objek utama yang diperiksa oleh sistem. Informasi seperti nama penulis, tahun publikasi, judul, nama jurnal, dan DOI akan diekstraksi dari dokumen dan digunakan untuk melakukan pemeriksaan terhadap metadata publikasi. Pemeriksaan tersebut bertujuan untuk mengidentifikasi referensi yang tidak dapat diverifikasi, metadata yang tidak sesuai, maupun DOI yang tidak valid.

2.2 Sitasi

Sitasi merupakan cara untuk memberikan pengakuan terhadap sumber yang digunakan dalam suatu karya ilmiah. Sitasi dapat muncul di dalam teks untuk menunjukkan sumber dari suatu informasi, gagasan, atau pernyataan yang digunakan oleh penulis. APA Style membedakan penggunaan sitasi dalam teks (*in-text citation*) dengan daftar referensi yang ditempatkan pada bagian akhir dokumen. Keduanya memiliki hubungan sehingga pembaca dapat menghubungkan sumber yang disebutkan dalam teks dengan informasi bibliografis lengkap pada daftar referensi (American Psychological Association, 2020).

Salah satu bentuk sitasi yang umum digunakan adalah sistem *author-date*, yaitu mencantumkan nama penulis dan tahun publikasi di dalam teks. Sebagai contoh, penulisan “(Koten, 2023)” menunjukkan bahwa informasi tersebut merujuk pada karya yang ditulis oleh Koten dan diterbitkan pada tahun 2023. Informasi tersebut kemudian seharusnya dapat ditemukan pada daftar referensi dengan informasi

bibliografis yang sesuai. Konsistensi antara sitasi dalam teks dan daftar referensi menjadi bagian penting dalam penulisan ilmiah karena memungkinkan pembaca melakukan penelusuran terhadap sumber yang digunakan.

Dalam penelitian ini, hubungan antara sitasi dan daftar pustaka digunakan sebagai dasar untuk melakukan *citation-reference matching*. Sistem akan mengekstraksi sitasi dari isi dokumen, kemudian mencocokkannya dengan entri daftar pustaka berdasarkan informasi seperti nama penulis dan tahun publikasi. Apabila suatu sitasi tidak memiliki pasangan yang sesuai pada daftar pustaka atau informasi penulis dan tahun tidak sesuai, sistem akan menandainya sebagai temuan yang perlu diperiksa lebih lanjut.

2.3 *Digital Object Identifier (DOI)*

Digital Object Identifier (DOI) merupakan pengenal unik dan persisten yang digunakan untuk mengidentifikasi suatu objek, termasuk publikasi ilmiah, dalam lingkungan digital. Sistem DOI dirancang agar suatu objek dapat diidentifikasi secara unik dan DOI tersebut dapat digunakan untuk mengarahkan pengguna menuju sumber atau informasi mengenai objek yang diidentifikasi. DOI Foundation menjelaskan bahwa setiap DOI memiliki metadata yang mendeskripsikan objek yang diwakilinya dan sistem DOI menyediakan mekanisme resolusi untuk menghubungkan DOI dengan sumber atau informasi terkait (International DOI Foundation, 2025).

Dalam publikasi ilmiah, DOI dapat digunakan untuk membantu pembaca menemukan dan mengakses karya yang dirujuk. DOI dapat ditampilkan dalam bentuk tautan “<https://doi.org/...>,” sehingga pengguna dapat melakukan resolusi DOI untuk menuju sumber yang berkaitan. Penggunaan DOI dalam referensi juga membantu mengurangi ketergantungan terhadap alamat halaman web tertentu yang dapat berubah dari waktu ke waktu (International DOI Foundation, 2025).

Dalam penelitian ini, DOI digunakan sebagai salah satu indikator untuk memverifikasi referensi ilmiah. Sistem akan mengambil DOI yang terdapat pada daftar pustaka dan melakukan pemeriksaan terhadap keberadaan DOI serta metadata publikasi yang terkait. Apabila DOI tidak ditemukan atau metadata yang diperoleh tidak sesuai dengan informasi pada daftar pustaka, sistem dapat menandainya sebagai temuan untuk dilakukan pemeriksaan lebih lanjut. Sebaliknya, apabila referensi tidak mencantumkan DOI tetapi metadata referensinya dapat digunakan untuk menemukan publikasi yang sesuai melalui sumber metadata, sistem dapat memberikan rekomendasi DOI kepada pengguna.

2.4 *Crossref API*

Crossref API merupakan antarmuka pemrograman yang digunakan untuk mengakses metadata publikasi ilmiah yang tersedia pada layanan *Crossref*. Metadata tersebut dapat mencakup informasi bibliografis seperti judul, penulis, tanggal publikasi, jurnal, DOI, referensi, serta berbagai pengenal lain yang berkaitan dengan suatu karya ilmiah. *Crossref* menyediakan metadata tersebut dalam format terstruktur yang dapat diakses oleh aplikasi melalui API, sehingga

memungkinkan proses pencarian dan pengolahan informasi bibliografis dilakukan secara otomatis.

Salah satu layanan utama yang disediakan *Crossref* adalah REST API. REST API memungkinkan sistem melakukan pencarian dan pengambilan metadata menggunakan berbagai *endpoint*, salah satunya “/works” untuk mencari karya ilmiah dan “/works/{doi}” untuk memperoleh metadata berdasarkan DOI tertentu. Hasil permintaan dapat dikembalikan dalam format JSON sehingga informasi yang diperoleh dapat langsung diproses oleh aplikasi. Selain itu, API menyediakan parameter pencarian dan filter yang memungkinkan hasil pencarian dibatasi berdasarkan atribut tertentu pada metadata.

Dalam penelitian ini, *Crossref API* digunakan sebagai sumber metadata pembanding dalam proses validasi daftar pustaka. Ketika sistem menemukan DOI pada suatu daftar pustaka, DOI tersebut dapat digunakan untuk mengambil metadata publikasi dari *Crossref*. Selanjutnya, informasi yang diperoleh dari *Crossref*, seperti penulis, judul, tahun publikasi, dan nama jurnal, dibandingkan dengan informasi yang diekstraksi dari daftar pustaka pada dokumen pengguna. Dengan demikian, keberadaan suatu DOI tidak hanya diperiksa berdasarkan apakah DOI tersebut terdaftar, tetapi juga dapat diperiksa kesesuaiannya dengan metadata referensi yang terdapat pada dokumen.

Crossref API juga dapat digunakan untuk melakukan pencarian metadata ketika DOI tidak tercantum pada daftar pustaka. Sistem dapat mengirimkan informasi bibliografis seperti judul atau penulis sebagai parameter pencarian untuk memperoleh beberapa kandidat publikasi dari *Crossref*. Kandidat tersebut kemudian dapat digunakan pada tahap pencocokan menggunakan metode *fuzzy matching* dan *semantic similarity* yang digunakan dalam penelitian. Oleh karena itu, *Crossref API* dalam penelitian ini berfungsi sebagai penyedia data eksternal, sedangkan penentuan apakah suatu referensi sesuai atau terindikasi bermasalah dilakukan oleh sistem berdasarkan hasil perbandingan metadata.

2.5 GROBID

GROBID (*GeneRation Of Bibliographic Data*) merupakan perangkat lunak yang digunakan untuk melakukan ekstraksi dan strukturisasi informasi dari dokumen ilmiah, khususnya dokumen dalam format PDF. GROBID menyediakan layanan untuk mengubah informasi yang terdapat dalam dokumen menjadi data terstruktur sehingga elemen-elemen seperti bagian dokumen, metadata publikasi, sitasi, dan daftar pustaka dapat diproses oleh aplikasi. GROBID menyediakan layanan berbasis REST API sehingga proses ekstraksi dapat diintegrasikan ke dalam aplikasi lain.

GROBID memiliki beberapa layanan untuk memproses dokumen ilmiah. Salah satu layanan yang relevan dengan penelitian ini adalah *processReferences*, yang digunakan untuk mengekstraksi seluruh daftar pustaka dari dokumen PDF dan mengubahnya menjadi struktur data dalam format *TEI XML* atau *BibTeX*. GROBID juga menyediakan layanan *processCitation* untuk memproses satu referensi bibliografis dalam bentuk teks dan mengubahnya menjadi data bibliografis yang

terstruktur. Hasil ekstraksi tersebut dapat memuat informasi seperti nama penulis, judul, tahun, jurnal, dan DOI yang kemudian dapat diproses oleh sistem.

Dalam penelitian ini, GROBID digunakan sebagai tahap ekstraksi informasi dari dokumen sebelum proses validasi referensi dan sitasi dilakukan. Dokumen yang diunggah pengguna diproses oleh GROBID untuk memperoleh informasi daftar pustaka dan sitasi dalam bentuk data terstruktur. Data tersebut kemudian digunakan sebagai masukan untuk proses pencarian metadata melalui *Crossref API* dan proses pencocokan menggunakan metode *fuzzy string matching* serta *semantic similarity*. Dengan demikian, GROBID berfungsi untuk memperoleh dan menstrukturkan informasi dari dokumen, bukan sebagai metode utama untuk menentukan apakah suatu referensi merupakan referensi yang valid atau terindikasi halu.

GROBID juga memiliki fitur *consolidation* yang memungkinkan metadata hasil ekstraksi dibandingkan atau diperkaya menggunakan sumber eksternal seperti *Crossref REST API*. Dalam proses konsolidasi, GROBID dapat menggunakan metadata yang diekstraksi dari dokumen untuk mencari informasi tambahan pada Crossref, termasuk DOI dan metadata publikasi. Namun, pada penelitian ini proses validasi dapat tetap dilakukan pada sistem utama agar hasil pencocokan dan penentuan status referensi dapat dikontrol menggunakan metode yang dirancang dalam penelitian.

2.6 Fuzzy Matching

Fuzzy matching merupakan pendekatan pencocokan dua atau lebih string yang memungkinkan adanya perbedaan karakter, susunan, atau penulisan di antara string yang dibandingkan. Berbeda dengan *exact matching* yang hanya menganggap dua string cocok apabila nilainya sama, *fuzzy matching* menghasilkan ukuran kemiripan atau jarak sehingga dua string yang memiliki perbedaan kecil masih dapat dianggap memiliki tingkat kemiripan tertentu. Pendekatan ini banyak digunakan dalam proses *record linkage*, pencarian data, dan pencocokan teks ketika terdapat kesalahan penulisan atau variasi representasi data (Pikies & Ali, 2021; Pîslaru et al., 2023).

Dalam penelitian ini, *fuzzy matching* digunakan untuk membantu mencocokkan informasi bibliografis yang berasal dari dokumen pengguna dengan metadata yang diperoleh dari *Crossref*. Dua algoritma yang digunakan adalah *Levenshtein Distance* dan *Jaro-Winkler Similarity*. Kedua algoritma tersebut sama-sama melakukan perbandingan berbasis karakter, tetapi memiliki karakteristik perhitungan yang berbeda sehingga dapat digunakan untuk menangani variasi penulisan pada informasi seperti nama penulis, judul, dan nama jurnal (Pikies & Ali, 2021; Pîslaru et al., 2023).

2.6.1. Levenshtein Distance

Levenshtein Distance merupakan algoritma yang mengukur jarak antara dua *string* berdasarkan jumlah minimum operasi perubahan karakter yang diperlukan untuk mengubah satu *string* menjadi *string* lainnya. Operasi tersebut terdiri dari penyisipan (*insertion*), penghapusan (*deletion*), dan penggantian (*substitution*)

sebuah karakter. Semakin kecil nilai *Levenshtein Distance*, semakin sedikit perubahan yang diperlukan untuk memperoleh *string* lainnya sehingga kedua *string* dapat dianggap semakin mirip (Pikies & Ali, 2021; Pîslaru et al., 2023).

Sebagai contoh, apabila terdapat dua nama penulis yaitu Kote dan Koton, perbedaan kedua *string* tersebut hanya terdapat pada satu karakter. *Levenshtein Distance* dapat memberikan nilai jarak yang rendah karena hanya diperlukan satu operasi penggantian untuk mengubah Kote menjadi Koton. Karakteristik tersebut membuat algoritma ini dapat digunakan untuk menangani kesalahan pengetikan atau perbedaan kecil pada data bibliografis.

Secara umum, *Levenshtein Distance* dapat dinyatakan dengan fungsi rekursif yang menghitung jarak minimum berdasarkan tiga kemungkinan operasi, yaitu penghapusan, penyisipan, dan penggantian karakter. Nilai akhirnya menunjukkan jarak perbedaan, sehingga semakin kecil nilainya maka semakin dekat kedua *string* yang dibandingkan. Penelitian mengenai *fuzzy string matching* juga menunjukkan bahwa *Levenshtein Distance* merupakan salah satu algoritma yang dapat digunakan untuk melakukan pencocokan *string* dalam sistem otomatis (Pikies & Ali, 2021).

Dalam penelitian ini, *Levenshtein Distance* dapat digunakan terutama untuk membandingkan nama jurnal, judul, atau informasi bibliografis yang memiliki kemungkinan kesalahan karakter. Nilai jarak yang diperoleh kemudian dapat dinormalisasi menjadi nilai kemiripan agar dapat dibandingkan dengan hasil algoritma lainnya. Hasil tersebut selanjutnya dapat menjadi salah satu komponen dalam menentukan apakah informasi pada daftar pustaka memiliki kemiripan dengan metadata yang diperoleh dari *Crossref*.

2.6.2. Jaro-Winkler

Jaro-Winkler Similarity merupakan algoritma pengukuran kemiripan *string* yang dikembangkan dari metode *Jaro*. Algoritma ini menghitung kemiripan berdasarkan karakter yang cocok, posisi karakter, dan transposisi karakter, kemudian memberikan peningkatan nilai ketika kedua *string* memiliki karakter awal (*prefix*) yang sama. Oleh karena itu, *Jaro-Winkler* dapat memberikan nilai kemiripan yang tinggi pada *string* yang memiliki kesamaan terutama pada bagian awalnya (Pîslaru et al., 2023; Microsoft, 2026).

Nilai *Jaro-Winkler* umumnya berada pada rentang 0 sampai 1, dengan nilai yang semakin mendekati 1 menunjukkan tingkat kemiripan yang semakin tinggi. Sebagai contoh, nama Kote dan Kote memiliki kemiripan penuh, sedangkan nama Kote dan Koton tetap dapat memperoleh nilai kemiripan yang tinggi karena sebagian besar karakter dan posisi karakter kedua *string* tersebut sama. Algoritma ini juga memberikan perhatian lebih terhadap kesamaan pada bagian awal *string* sehingga sesuai untuk data yang sering memiliki pola penulisan serupa pada awal teks (Pîslaru et al., 2023; Apache Software Foundation, 2026).

Karakteristik tersebut membuat *Jaro-Winkler* banyak digunakan pada permasalahan *string matching*, khususnya ketika data yang dibandingkan berupa nama atau teks pendek. Penelitian Pîslaru et al. (2023), menggunakan

Jaro-Winkler untuk melakukan pencocokan string dalam jumlah data yang besar dan membahas karakteristik serta efisiensi algoritma tersebut. Implementasi *Jaro-Winkler* pada perangkat lunak modern juga menunjukkan bahwa algoritma ini menghasilkan nilai kemiripan berdasarkan persentase karakter yang cocok serta memberikan bobot tambahan terhadap karakter awal yang sama (Pişlaru et al., 2023).

Dalam penelitian ini, *Jaro-Winkler* dapat digunakan untuk membandingkan nama penulis antara daftar pustaka pada dokumen dengan metadata yang diperoleh dari *Crossref*. Hal ini berguna ketika terdapat variasi kecil dalam penulisan nama, seperti kesalahan satu karakter, perbedaan penulisan, atau variasi format nama. Nilai kemiripan yang dihasilkan kemudian dapat dikombinasikan dengan hasil pencocokan atribut lain, seperti judul, tahun, dan nama jurnal, untuk menentukan tingkat kesesuaian suatu referensi.

2.7 *Sentence-BERT* (SBERT)

Sentence-BERT (SBERT) merupakan pendekatan berbasis *Transformer* yang digunakan untuk menghasilkan representasi vektor (*embedding*) dari suatu kalimat atau teks. Representasi tersebut memungkinkan teks diubah menjadi vektor berdimensi tetap sehingga tingkat kemiripan antara dua teks dapat dihitung secara lebih efisien. *Model Sentence Transformer* dapat digunakan untuk berbagai tugas, seperti *semantic textual similarity*, *semantic search*, *clustering*, klasifikasi, dan *paraphrase mining* (Guder et al., 2026).

Pada prosesnya, SBERT menghasilkan *embedding* untuk masing-masing teks yang dibandingkan. Dua teks yang memiliki makna semantik yang dekat diharapkan menghasilkan representasi vektor yang memiliki tingkat kemiripan tinggi, meskipun susunan kata yang digunakan tidak sepenuhnya sama. Pendekatan berbasis *sentence embedding* ini memungkinkan sistem melakukan perbandingan berdasarkan makna teks, bukan hanya berdasarkan kesamaan karakter atau kata secara langsung (Guder et al., 2026; Sentence Transformers, 2026b).

Salah satu cara yang dapat digunakan untuk membandingkan *embedding* adalah *Cosine Similarity*. Metode ini menghitung kedekatan dua vektor berdasarkan sudut di antara keduanya, sehingga semakin dekat arah kedua vektor, semakin tinggi nilai kemiripannya. Dokumentasi *Sentence Transformers* menjelaskan bahwa *semantic textual similarity* dapat dilakukan dengan menghasilkan *embedding* dari teks kemudian menghitung nilai kemiripan antar-*embedding*, dengan *cosine similarity* sebagai salah satu metode yang tersedia dan menjadi pilihan bawaan pada implementasi tertentu.

Dalam penelitian ini, SBERT digunakan untuk membandingkan kemiripan semantik judul referensi yang diperoleh dari dokumen pengguna dengan judul publikasi yang diperoleh dari *Crossref*. Sebagai contoh, judul pada daftar pustaka dapat memiliki perbedaan susunan kata atau sedikit perbedaan penulisan dibandingkan metadata *Crossref*, tetapi tetap merujuk pada publikasi yang sama. Dengan menggunakan *sentence embedding*, sistem dapat memperoleh nilai kemiripan semantik antara kedua judul tersebut dan menggunakan nilai tersebut

sebagai salah satu komponen dalam proses validasi referensi. Pendekatan ini melengkapi *fuzzy matching* seperti *Levenshtein Distance* dan *Jaro-Winkler* yang lebih berfokus pada kemiripan berbasis karakter.

Dalam penerapannya, nilai kemiripan SBERT tidak sebaiknya langsung dianggap sebagai bukti bahwa dua referensi adalah publikasi yang sama. Nilai tersebut lebih tepat digunakan sebagai salah satu indikator yang dikombinasikan dengan informasi lain, seperti kesesuaian penulis, tahun publikasi, nama jurnal, dan DOI. Dengan demikian, penelitian ini dapat menggunakan SBERT untuk menangkap kesamaan makna judul, sementara atribut bibliografis lainnya digunakan sebagai informasi pendukung dalam menentukan kesesuaian referensi.

2.8 Evaluasi Sistem Deteksi

Evaluasi sistem deteksi dilakukan untuk mengukur kemampuan sistem dalam membedakan data yang termasuk dalam kategori valid dan tidak valid berdasarkan ground truth. Pada penelitian yang berkaitan dengan ekstraksi dan pencocokan sitasi, precision dan recall digunakan untuk melihat keseimbangan antara kesalahan berupa data yang salah ditandai dan data yang seharusnya terdeteksi tetapi terlewatkan. Marx et al. (2022), misalnya, menggunakan *precision* dan *recall* untuk mengevaluasi proses ekstraksi dan pencocokan sitasi serta menunjukkan bahwa kedua metrik tersebut perlu dilihat secara bersamaan karena peningkatan salah satu metrik dapat diikuti penurunan metrik lainnya. Penelitian mengenai integritas sitasi juga menggunakan *precision*, *recall*, dan *F1-score* untuk mengevaluasi kemampuan model dalam mengidentifikasi informasi yang berkaitan dengan sitasi ilmiah (Marx et al., 2022).

Pada penelitian ini, evaluasi dilakukan dengan membandingkan hasil deteksi sistem dengan *ground truth* yang telah ditentukan sebelumnya. *Confusion matrix* digunakan untuk melihat distribusi prediksi benar dan kesalahan prediksi, sedangkan *precision*, *recall*, dan *F1-score* digunakan sebagai metrik kuantitatif untuk mengukur performa deteksi. Pendekatan tersebut relevan karena sistem yang dikembangkan tidak hanya perlu menghasilkan prediksi yang benar, tetapi juga perlu mampu menemukan sebanyak mungkin referensi atau sitasi yang memang bermasalah. Penggunaan metrik tersebut juga umum pada penelitian klasifikasi dan ekstraksi informasi berbasis NLP, termasuk penelitian yang berkaitan dengan sitasi ilmiah.

2.8.1. Confusion Matrix

Confusion matrix merupakan tabel yang digunakan untuk membandingkan hasil prediksi sistem dengan kondisi sebenarnya dari data. Pada klasifikasi biner, confusion matrix terdiri atas *True Positive (TP)*, *True Negative (TN)*, *False Positive (FP)*, dan *False Negative (FN)*. TP menunjukkan data positif yang berhasil dikenali sebagai positif, sedangkan TN menunjukkan data negatif yang berhasil dikenali sebagai negatif. Sementara itu, FP menunjukkan data negatif yang salah diklasifikasikan sebagai positif dan FN menunjukkan data positif yang gagal dikenali oleh sistem (Marx et al., 2022; Kokol et al., 2024).

Confusion matrix sesuai digunakan dalam penelitian ini karena sistem yang dikembangkan memiliki tugas klasifikasi terhadap kondisi referensi atau sitasi. Sebagai contoh, apabila kelas positif ditetapkan sebagai referensi tidak valid, maka TP merupakan jumlah referensi tidak valid yang berhasil dideteksi sistem, sedangkan FN merupakan referensi tidak valid yang tidak berhasil ditemukan. FP menunjukkan referensi valid yang secara keliru ditandai sebagai tidak valid. Dengan melihat keempat komponen tersebut, kesalahan sistem dapat dianalisis secara lebih rinci sebelum dihitung menjadi precision, recall, dan F1-score.

2.8.2. Precision

Precision merupakan ukuran yang menunjukkan seberapa banyak hasil yang diprediksi sebagai kelas positif benar-benar merupakan anggota kelas positif. *Precision* dihitung dengan membandingkan jumlah *True Positive* dengan keseluruhan data yang diprediksi positif, yaitu *True Positive* dan *False Positive*. Dengan demikian, *precision* berhubungan dengan kemampuan sistem untuk menghindari *false positive*, yaitu kondisi ketika sistem menandai data yang sebenarnya benar sebagai data bermasalah (Marx et al., 2022; Kokol et al., 2024).

$$Precision = \frac{TP}{TP + FP}$$

Dalam penelitian ini, precision penting karena sistem akan memberikan tanda atau highlight terhadap referensi dan sitasi yang dianggap tidak valid. *Precision* yang rendah dapat menyebabkan banyak referensi yang sebenarnya valid ikut ditandai sebagai bermasalah. Oleh karena itu, precision digunakan untuk mengetahui ketepatan sistem ketika memberikan hasil deteksi positif. Penggunaan *precision* dalam penelitian ekstraksi dan pencocokan sitasi juga telah diterapkan untuk mengetahui seberapa banyak hasil yang dinyatakan sebagai suatu kecocokan benar-benar sesuai dengan kondisi sebenarnya (Marx et al., 2022).

2.8.3. Recall

Recall merupakan ukuran yang menunjukkan kemampuan sistem dalam menemukan seluruh data yang sebenarnya termasuk dalam kelas positif. *Recall* dihitung berdasarkan jumlah *True Positive* dibandingkan dengan keseluruhan data positif sebenarnya, yaitu *True Positive* dan *False Negative*. Dengan demikian, *recall* berfokus pada kemampuan sistem dalam mengurangi *false negative*, yaitu kondisi ketika data yang sebenarnya bermasalah tidak berhasil dideteksi oleh sistem (Marx et al., 2022; Kokol et al., 2024).

$$Recall = \frac{TP}{TP + FN}$$

Recall menjadi penting dalam penelitian ini karena salah satu tujuan sistem adalah menemukan referensi dan sitasi yang tidak valid. Apabila sistem memiliki *recall* rendah, sebagian referensi atau sitasi yang sebenarnya bermasalah dapat

lolos dari proses pemeriksaan. Sebaliknya, *recall* yang tinggi menunjukkan bahwa sebagian besar data bermasalah yang terdapat pada ground truth berhasil ditemukan oleh sistem. Pada penelitian ekstraksi dan resolusi sitasi, *precision* dan *recall* juga digunakan secara bersamaan untuk mengevaluasi kemampuan sistem menemukan kecocokan yang benar dan mengurangi kecocokan yang terlewatkan (Marx et al., 2022).

2.8.4. F1-Score

F1-score merupakan metrik yang menggabungkan *precision* dan *recall* menggunakan *harmonic mean*. *F1-score* digunakan untuk memberikan ukuran tunggal yang mempertimbangkan kedua aspek tersebut secara bersamaan. Metrik ini menjadi relevan ketika suatu sistem perlu mempertahankan ketepatan prediksi sekaligus kemampuan menemukan sebanyak mungkin data positif (Kokol et al., 2024).

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

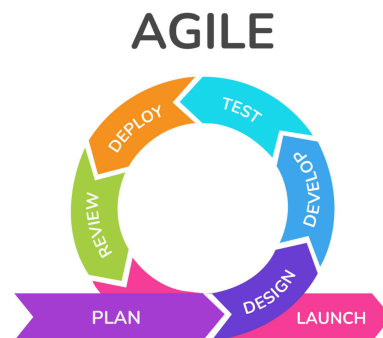
Dalam penelitian ini, *F1-score* digunakan untuk memberikan gambaran keseimbangan antara ketepatan sistem dalam menandai referensi/sitasi bermasalah dan kemampuan sistem dalam menemukan referensi/sitasi bermasalah yang sebenarnya. Hal tersebut penting karena sistem yang hanya berfokus pada *precision* dapat menghasilkan sedikit deteksi agar kesalahan *false positive* rendah, sementara sistem yang hanya berfokus pada *recall* dapat menandai terlalu banyak referensi yang sebenarnya valid. Oleh karena itu, *F1-score* dapat digunakan sebagai salah satu metrik utama untuk mengevaluasi performa keseluruhan metode deteksi yang dikembangkan. Penelitian mengenai integritas sitasi juga menggunakan *F1-score* untuk mengevaluasi kemampuan sistem dalam mengidentifikasi dan mengklasifikasikan informasi terkait sitasi ilmiah

2.9 Agile

Agile merupakan pendekatan dalam pengembangan perangkat lunak yang menekankan kemampuan beradaptasi terhadap perubahan, pengembangan secara iteratif, serta kolaborasi selama proses pengembangan. Berbeda dengan pendekatan yang menempatkan seluruh kebutuhan sistem sebagai sesuatu yang harus ditentukan sejak awal, Agile memungkinkan kebutuhan dan solusi dikembangkan secara bertahap melalui beberapa iterasi. Kajian sistematis mengenai Agile menunjukkan bahwa pendekatan ini banyak digunakan karena kemampuannya dalam menyesuaikan proses pengembangan terhadap perubahan kebutuhan dan kondisi proyek (Ozkan & Gok, 2022; Edison et al., 2022).

Dalam Agile, pengembangan perangkat lunak dilakukan melalui siklus atau iterasi yang memungkinkan tim menghasilkan bagian tertentu dari sistem, melakukan evaluasi, kemudian melakukan perbaikan pada iterasi berikutnya. Setiap iterasi dapat menghasilkan peningkatan terhadap perangkat lunak

berdasarkan hasil evaluasi dan kebutuhan yang ditemukan selama proses pengembangan. Pendekatan iteratif tersebut juga memungkinkan metode Agile disesuaikan dengan karakteristik dan kebutuhan suatu proyek, karena penelitian mengenai implementasi Scrum menunjukkan bahwa praktik Agile sering disesuaikan dengan konteks proyek yang dihadapi (Dikert et al., 2022; Edison et al., 2022).



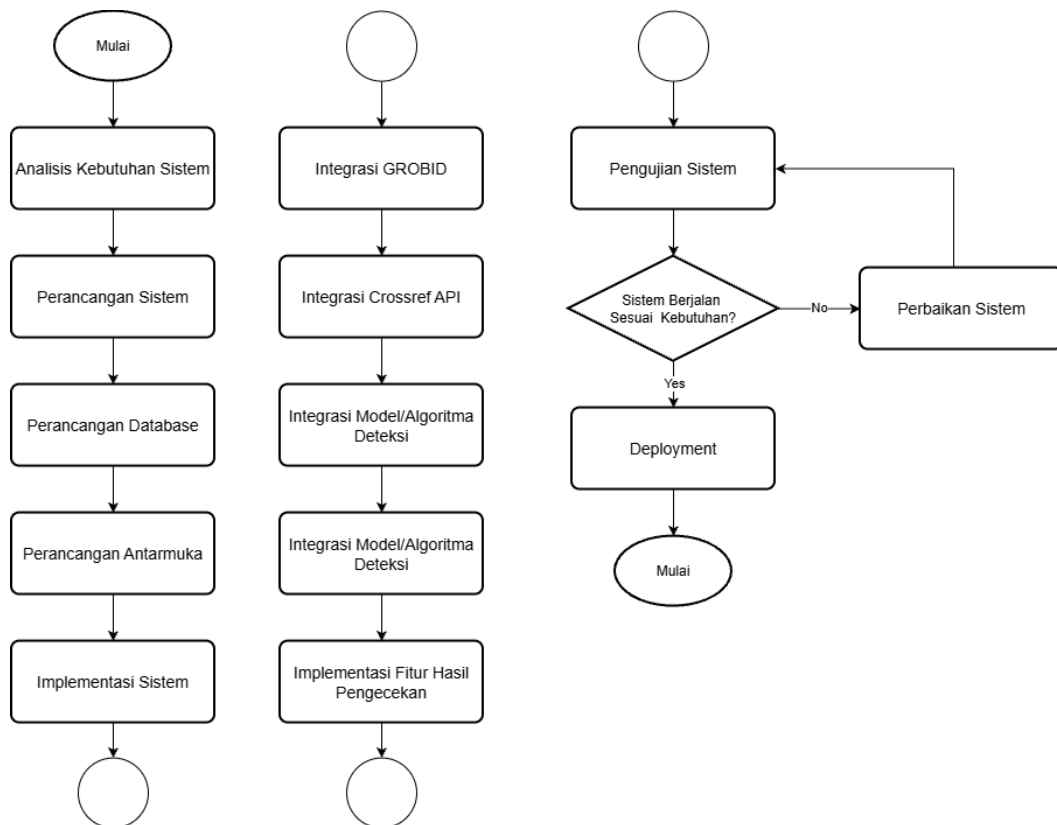
Gambar 2.9 Tahapan Agile

Dalam penelitian ini, Agile digunakan sebagai metode pengembangan perangkat lunak untuk membangun sistem deteksi daftar pustaka dan sitasi. Penggunaan Agile memungkinkan pengembangan sistem dilakukan secara bertahap, mulai dari fungsi dasar seperti unggah dokumen dan ekstraksi referensi, kemudian dilanjutkan dengan integrasi Crossref API, penerapan fuzzy matching dan SBERT, hingga visualisasi hasil deteksi. Pendekatan bertahap tersebut sesuai dengan karakteristik penelitian karena beberapa komponen sistem, khususnya metode pencocokan dan batas nilai kemiripan (threshold), kemungkinan memerlukan pengujian dan penyesuaian berdasarkan hasil eksperimen. Penelitian mengenai Agile juga menunjukkan bahwa metode tersebut dapat disesuaikan dengan karakteristik proyek dan kebutuhan pengembangan yang berbeda (Dikert et al., 2022; Trihardianingsih et al., 2023)

BAB 3 METODOLOGI

3.1 Diagram Alir Proyek

3.1.1. Diagram Alir Pembuatan Sistem



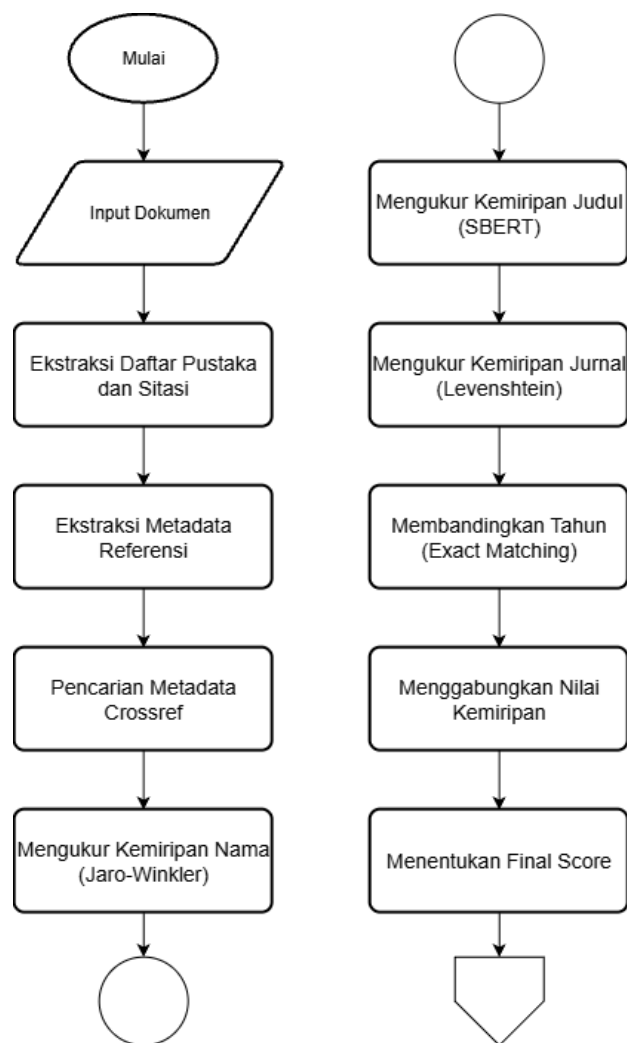
Gambar 3.1.1 Diagram Alir Pembuatan Sistem

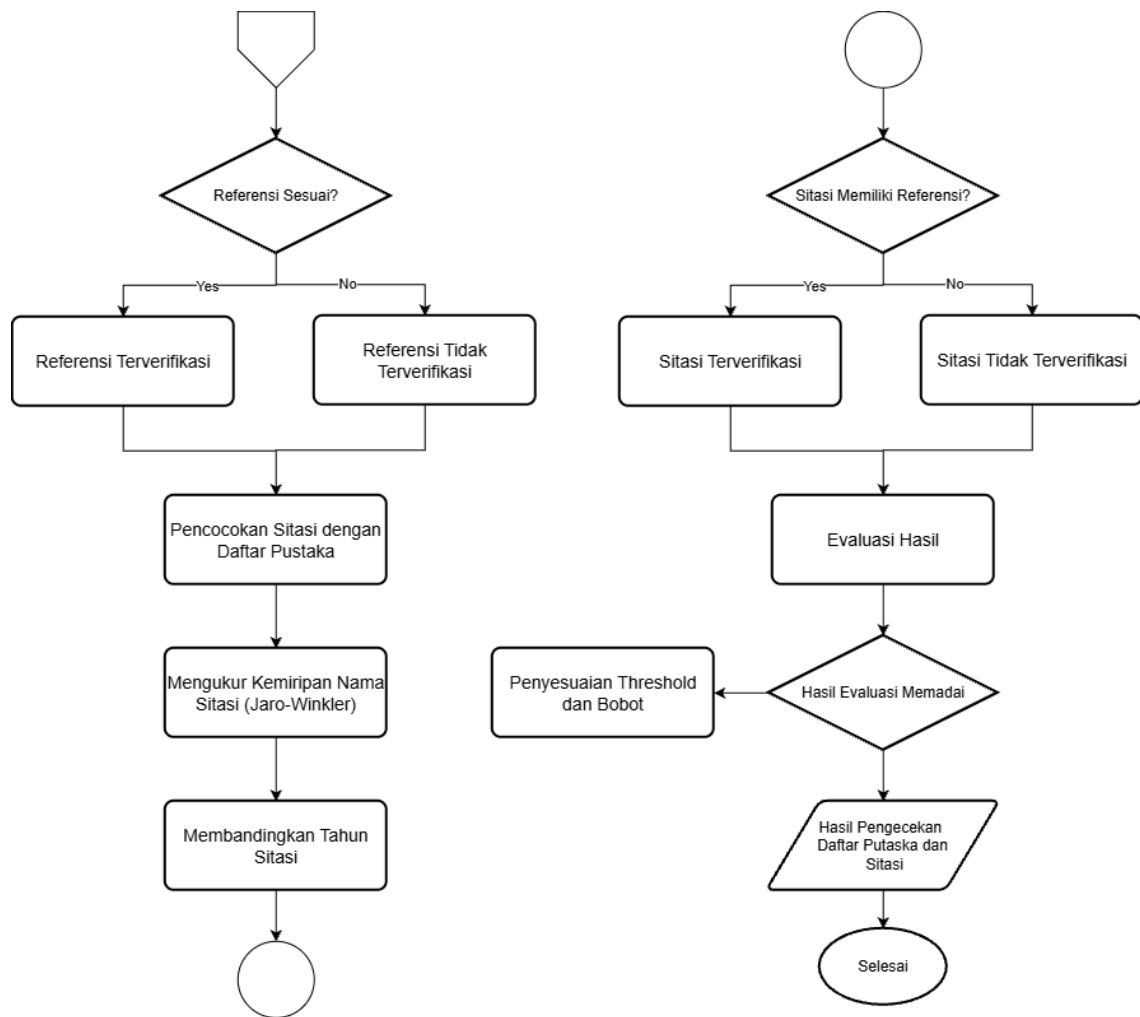
Tahap pembuatan sistem diawali dengan analisis kebutuhan sistem untuk menentukan kebutuhan dan fitur yang diperlukan dalam proses pemeriksaan daftar pustaka dan sitasi. Selanjutnya dilakukan perancangan sistem, database, dan antarmuka untuk menentukan alur kerja sistem, struktur penyimpanan data, serta tampilan yang akan digunakan oleh pengguna. Setelah rancangan selesai, dilakukan implementasi sistem dengan membangun fungsi utama seperti pengunggahan dokumen, pengelolaan data, dan penyimpanan hasil pemeriksaan. Sistem kemudian diintegrasikan dengan GROBID untuk mengekstraksi informasi daftar pustaka dan sitasi dari dokumen, *Crossref API* untuk memperoleh serta memvalidasi metadata publikasi, dan model/algoritma deteksi untuk membandingkan informasi referensi dengan metadata yang diperoleh. Setelah seluruh komponen terintegrasi, dikembangkan fitur hasil pengecekan untuk menampilkan status referensi dan sitasi serta memberikan penandaan pada bagian dokumen yang terindikasi bermasalah.

Sistem yang telah selesai dikembangkan selanjutnya masuk ke tahap pengujian sistem untuk memastikan setiap fungsi berjalan sesuai dengan kebutuhan yang

telah ditentukan. Pengujian dilakukan pada proses utama, mulai dari pengunggahan dokumen, ekstraksi informasi, validasi dan pencarian referensi, pemeriksaan sitasi, hingga penyimpanan dan penampilan hasil pengecekan. Jika ditemukan kesalahan atau hasil yang belum sesuai, dilakukan perbaikan sistem pada bagian yang bermasalah, kemudian sistem diuji kembali untuk memastikan perbaikan telah berhasil. Proses pengujian dan perbaikan dilakukan hingga sistem memenuhi kebutuhan yang telah ditetapkan. Setelah sistem dinyatakan berjalan sesuai kebutuhan, sistem dilanjutkan ke tahap deployment agar dapat digunakan untuk melakukan pemeriksaan dokumen.

3.1.2. Diagram Alir Pembuatan Model AI Deteksi





Gambar 3.1.2 Diagram Alir Pembuatan Model AI Deteksi

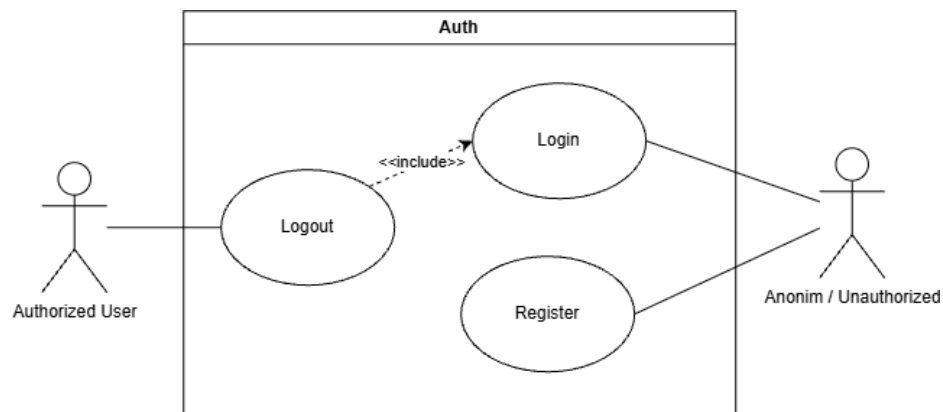
Proses diawali dengan input dokumen yang akan diperiksa, kemudian dilakukan ekstraksi daftar pustaka dan sitasi untuk memperoleh informasi yang akan dianalisis. Pada daftar pustaka, sistem melakukan ekstraksi metadata referensi seperti nama penulis, judul, jurnal, tahun, dan DOI, kemudian melakukan pencarian metadata melalui *Crossref* sebagai data pembandingan. Selanjutnya, sistem mengukur kemiripan nama menggunakan *Jaro-Winkler*, kemiripan judul menggunakan SBERT, dan kemiripan jurnal menggunakan *Levenshtein*, serta melakukan perbandingan tahun menggunakan *Exact Matching*. Nilai dari seluruh proses tersebut kemudian digabungkan untuk memperoleh *Final Score* yang digunakan dalam menentukan apakah suatu referensi sesuai atau tidak dengan metadata yang ditemukan dari *Crossref*.

Setelah proses pemeriksaan referensi selesai, sistem melakukan pencocokan sitasi dengan daftar pustaka untuk memastikan setiap sitasi yang terdapat dalam teks memiliki referensi yang sesuai. Pada tahap ini, sistem mengukur kemiripan nama sitasi menggunakan *Jaro-Winkler* dan membandingkan tahun sitasi dengan tahun pada daftar pustaka untuk menentukan apakah sitasi memiliki referensi yang sesuai. Hasil pemeriksaan kemudian masuk ke tahap evaluasi hasil untuk

mengukur kemampuan algoritma dalam mendeteksi referensi dan sitasi yang sesuai maupun tidak sesuai. Jika hasil evaluasi belum memadai, dilakukan penyesuaian threshold dan bobot kemudian proses penggabungan nilai kemiripan dilakukan kembali. Setelah diperoleh hasil evaluasi yang memadai, algoritma digunakan untuk menghasilkan hasil pengecekan daftar pustaka dan sitasi pada dokumen.

3.2 Use Case Diagram

3.2.1. Use Case Authentication



Gambar 3.2.1. Use Case Authion

Use Case Diagram Auth menggambarkan proses autentikasi pengguna dalam sistem, yang melibatkan dua jenis aktor yaitu *Unauthorized User* dan *Authorized User*. *Unauthorized User* memiliki akses untuk melakukan registrasi (*Register*) dan *login* (*Login*) ke dalam sistem. Setelah berhasil *login*, pengguna akan menjadi *Authorized User* yang dapat mengakses fitur *logout* (*Logout*) untuk mengakhiri sesi penggunaan.

Terdapat relasi *include* antara *logout* dan *login* yang menunjukkan bahwa proses *logout* hanya dapat dilakukan jika pengguna telah melalui proses *login* sebelumnya. Diagram ini menegaskan bahwa *autentikasi* merupakan gerbang utama dalam sistem, di mana hanya pengguna yang telah terverifikasi yang dapat mengakses fitur utama aplikasi, sehingga aspek keamanan dan kontrol akses dapat terjaga dengan baik.

3.2.2. Use Case System



Gambar 3.2.2. Use Case System

Diagram ini menggambarkan seluruh kemampuan yang dimiliki oleh sistem, yang melibatkan dua pihak: *Authorized User* (pengguna yang sudah login ke dalam sistem) dan *API Eksternal* (layanan pihak ketiga yang dipakai sistem untuk memverifikasi data referensi).

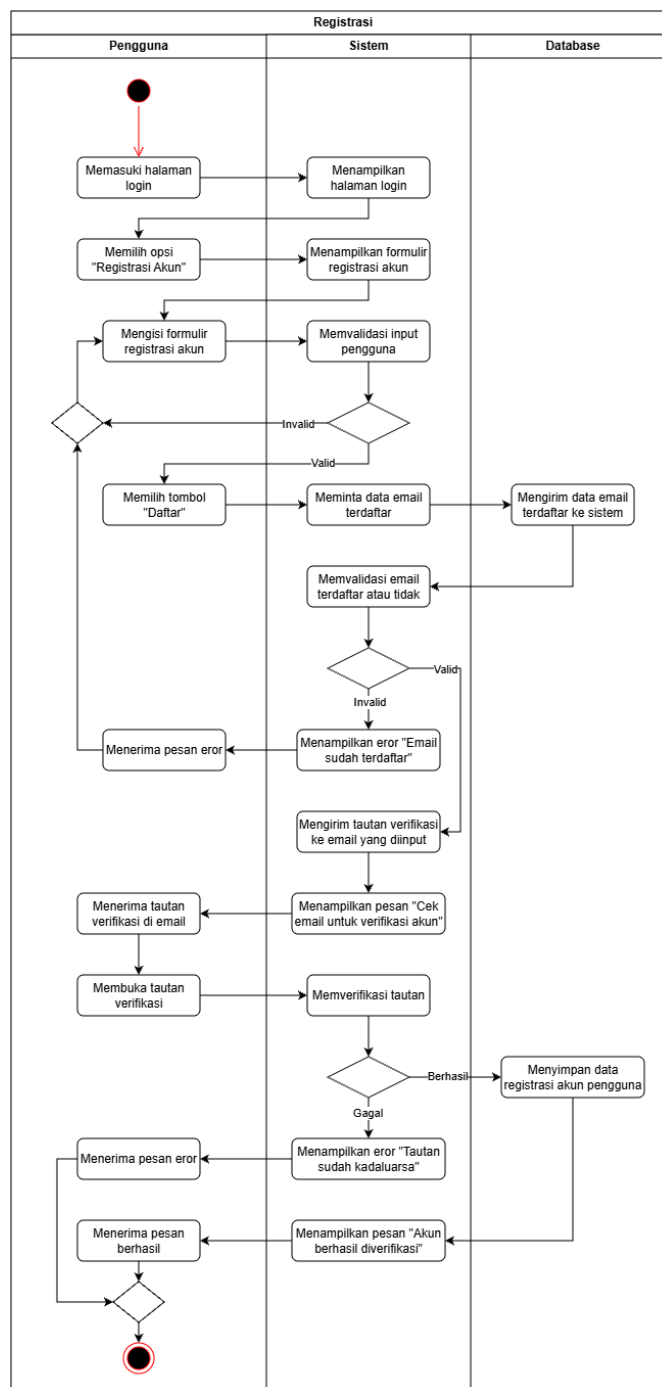
Sebagai pengguna, *Authorized User* dapat mengunggah dokumen ke dalam sistem, lalu menjalankan proses pengecekan melalui *use case* Mengecek Dokumen. Setelah dokumen diperiksa, pengguna dapat melihat hasil pengecekan dokumen yang sudah selesai diproses, atau kembali membuka pemeriksaan sebelumnya lewat melihat riwayat dokumen. Kalau riwayat itu sudah tidak diperlukan, pengguna juga bisa menghapus riwayat dokumen tersebut. Selain itu, pengguna dapat mengunduh hasil dokumen yang telah dicek dalam bentuk laporan, serta merevisi hasil pengecekan apabila ada temuan sistem yang dirasa kurang tepat dan perlu dikoreksi secara manual.

Bagian paling inti dari sistem ini ada pada *use case* Mengecek Dokumen, yang secara otomatis menjalankan dua proses deteksi di baliknya (ditandai relasi <<include>>, artinya kedua proses ini pasti dijalankan setiap kali dokumen diperiksa). Proses pertama adalah Mendeteksi Sitasi Tidak Valid/Halu, yaitu memeriksa apakah setiap sitasi yang ditulis di dalam naskah (misalnya "(Nama, Tahun)") benar-benar memiliki pasangan yang sesuai di daftar pustaka. Proses kedua adalah Mendeteksi Daftar Pustaka Tidak Valid/Halu, yang memeriksa keabsahan setiap entri referensi di daftar pustaka.

Proses deteksi daftar pustaka ini kemudian menyertakan pemeriksaan lebih lanjut, yaitu Mendeteksi DOI pada Daftar Pustaka, yang bertugas mengecek apakah DOI yang tercantum pada suatu referensi benar-benar valid. Proses ini melibatkan API Eksternal karena sistem perlu menghubungi basis data pihak ketiga untuk memastikan DOI tersebut memang ada dan sesuai. Dari proses deteksi DOI ini ada *use case* tambahan yang sifatnya kondisional, yaitu Mencari DOI yang sesuai dengan Daftar Pustaka, yang dihubungkan dengan relasi <<extend>>. Artinya, *use case* pencarian DOI ini tidak selalu dijalankan, melainkan hanya muncul pada kondisi tertentu, contohnya ketika referensi terdeteksi sudah benar formatnya tetapi belum memiliki DOI, sehingga sistem baru mencoba mencari DOI yang cocok untuk referensi tersebut.

3.3 Activity Diagram

3.3.1. Activity Register



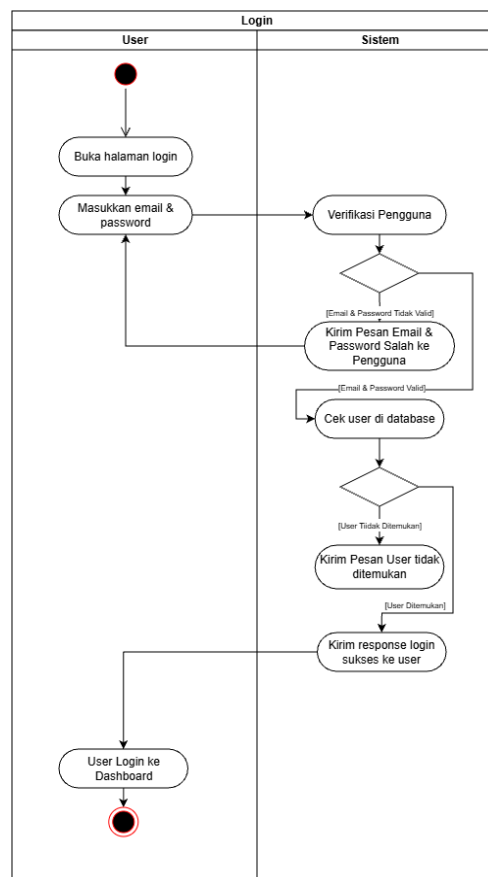
Gambar 3.3.1. Activity Register

Diagram dimulai ketika pengguna memasuki halaman login, yang memicu sistem untuk menampilkan antarmuka masuk sebelum pengguna memilih opsi "Registrasi Akun" guna mengakses formulir pendaftaran. Setelah pengguna mengisi data pada formulir, sistem segera melakukan validasi input untuk memastikan kesesuaian format; jika data dinyatakan *invalid*, pengguna diminta

untuk mengisi ulang, namun jika valid, pengguna dapat memilih tombol "Daftar" untuk melanjutkan proses. Pada tahap ini, sistem akan meminta data email terdaftar kepada *database* untuk memverifikasi apakah email tersebut sudah pernah digunakan atau tidak. Apabila email ternyata sudah terdaftar, sistem akan menampilkan pesan error "Email sudah terdaftar", namun jika email dinyatakan tersedia, sistem akan mengirimkan tautan verifikasi ke alamat email yang diinputkan dan meminta pengguna untuk melakukan pengecekan.

Setelah pengguna menerima dan membuka tautan verifikasi tersebut, sistem akan melakukan proses verifikasi tautan guna memastikan keabsahan dan masa berlakunya. Jika verifikasi gagal karena tautan sudah kadaluarsa, sistem akan menampilkan pesan *error* kepada pengguna, namun jika proses verifikasi berhasil, database akan segera menyimpan data registrasi akun pengguna secara permanen. Aktivitas berakhir ketika sistem menampilkan pesan konfirmasi bahwa akun telah berhasil diverifikasi, sehingga pengguna menerima notifikasi sukses sebagai tanda bahwa proses registrasi telah selesai dilakukan sepenuhnya hingga mencapai titik terminal akhir.

3.3.2. Activity Login

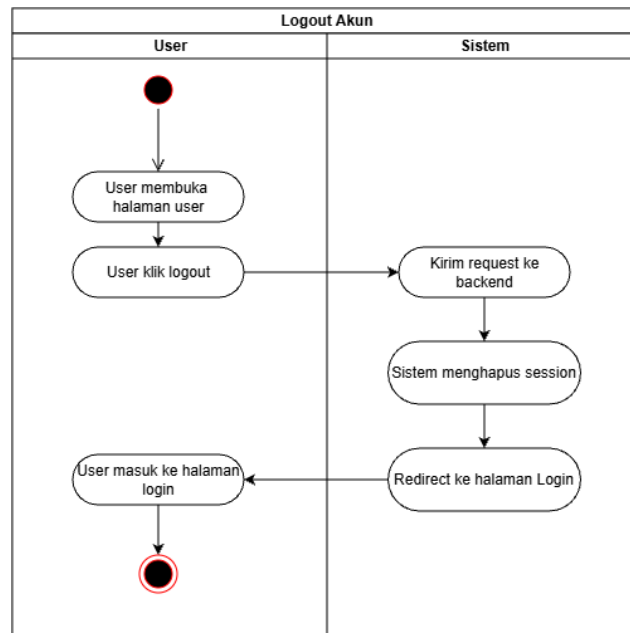


Gambar 3.3.2. Activity Login

Diagram dimulai ketika pengguna membuka halaman login dan memasukkan email serta password. Input tersebut dikirim ke sistem untuk diverifikasi. Sistem

pertama-tama mengecek apakah format email dan password valid. Jika tidak valid, sistem mengirim pesan *error* ke pengguna dan pengguna diminta mengisi ulang. Jika format valid, sistem melanjutkan dengan mencari akun di *database*. Jika user tidak ditemukan, sistem mengirim pesan bahwa akun tidak ada. Jika user ditemukan, sistem mengirim *response login* sukses dan pengguna diarahkan masuk ke *dashboard*.

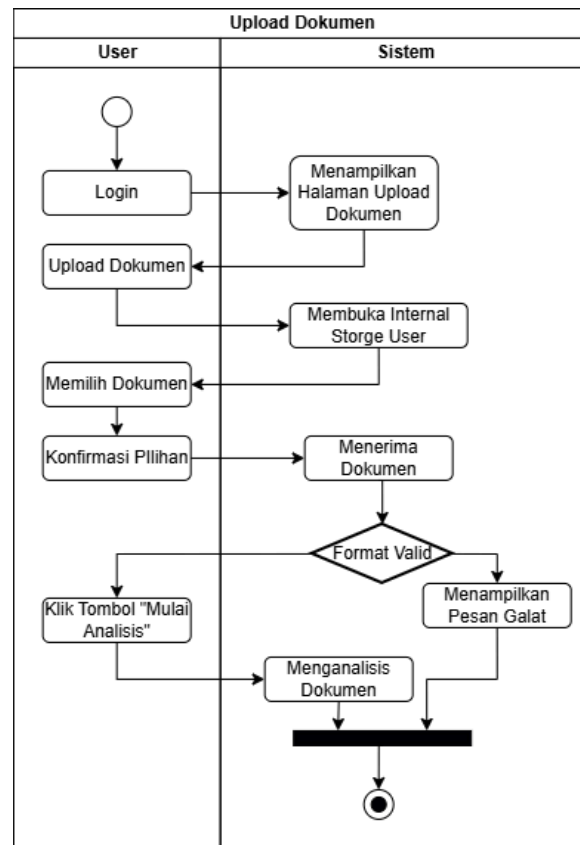
3.3.3. Activity Logout



Gambar 3.3.3. Activity Logout

Diagram dimulai ketika pengguna membuka halaman profil/user. Pengguna menekan tombol *logout*, lalu *frontend* mengirim *request* ke *backend*. *Backend* memproses penghapusan *session*, kemudian melakukan *redirect* ke halaman *login*.

3.3.4. Activity Mengunggah Dokumen

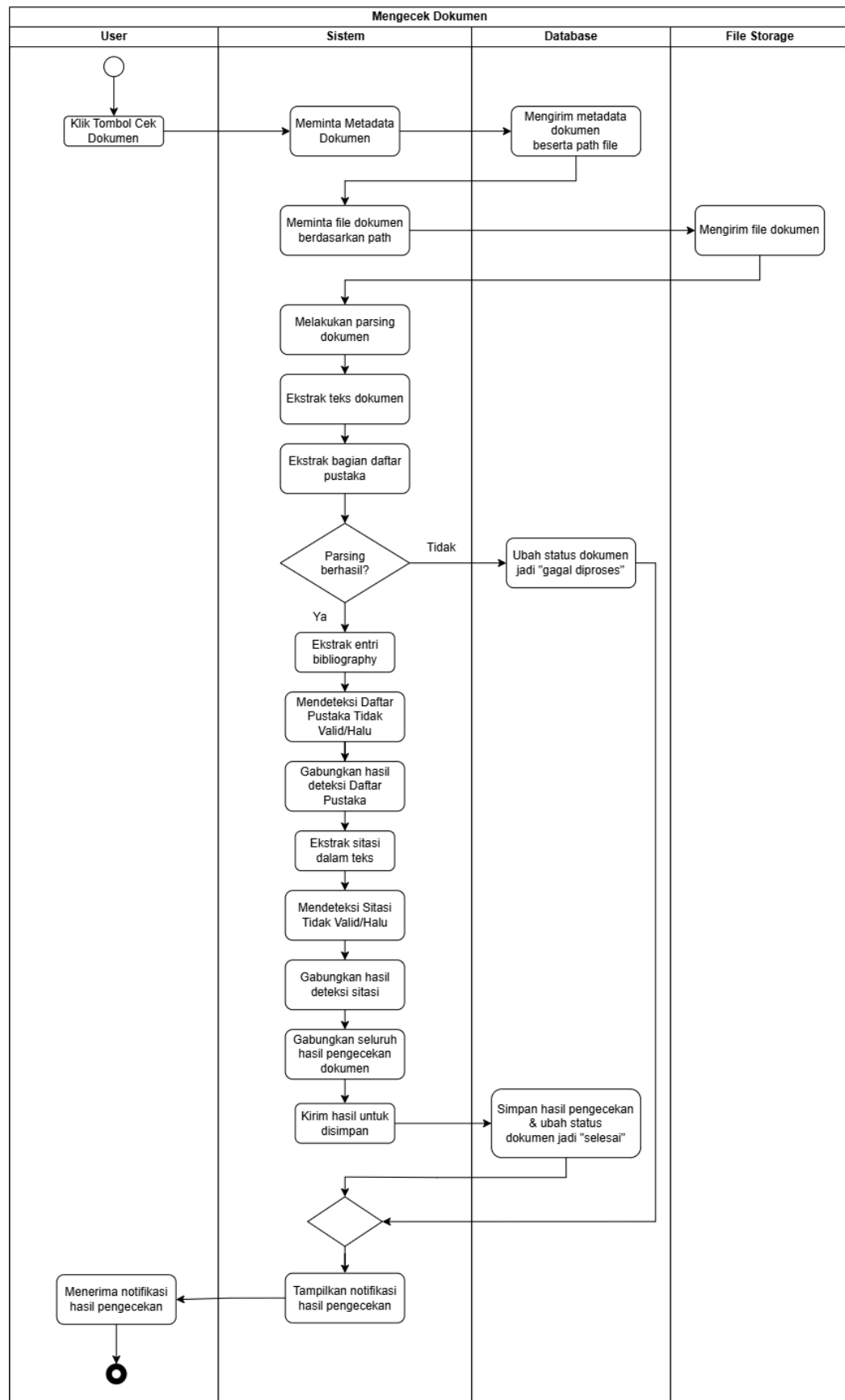


Gambar 3.3.4. Activity Mengunggah Dokumen

Diagram aktivitas ini menggambarkan alur proses "Upload Dokumen" pada sistem deteksi daftar pustaka baru, yang melibatkan interaksi antara *User* dan Sistem mulai dari tahap login hingga proses analisis dokumen dimulai. Proses diawali pada sisi *User*, di mana pengguna terlebih dahulu melakukan aktivitas "*Login*" untuk masuk ke dalam sistem. Setelah *login* berhasil, kendali berpindah ke sisi Sistem yang menjalankan aktivitas "*Menampilkan Halaman Upload Dokumen*" sebagai halaman antarmuka bagi pengguna untuk mengunggah dokumen yang akan diperiksa daftar pustakanya. Selanjutnya, pengguna melakukan aksi "*Upload Dokumen*", yang direspons oleh Sistem melalui aktivitas "*Membuka Internal Storage User*" untuk mengakses penyimpanan lokal milik pengguna. Pada tahap ini, pengguna kemudian melakukan aktivitas "*Memilih Dokumen*" untuk menentukan *file* yang ingin diunggah, dan dilanjutkan dengan "*Konfirmasi Pilihan*" sebagai penegasan bahwa dokumen yang dipilih sudah sesuai. Dokumen yang telah dikonfirmasi tersebut selanjutnya diproses oleh Sistem melalui aktivitas "*Menerima Dokumen*", yang kemudian dilanjutkan dengan sebuah *decision node* bernama "*Format Valid*" untuk memeriksa apakah format dokumen yang diunggah sesuai dengan ketentuan sistem atau tidak. Apabila format dokumen tidak valid, sistem akan menjalankan aktivitas "*Menampilkan Pesan Galat*" untuk memberi tahu pengguna bahwa dokumen yang

diunggah tidak dapat diproses, sehingga pengguna perlu mengunggah ulang dokumen dengan format yang sesuai. Sebaliknya, apabila format dokumen dinyatakan valid, kendali kembali ke sisi *User*, di mana pengguna melakukan aksi "Klik Tombol Mulai Analisis" untuk memulai proses pemeriksaan daftar pustaka. Aksi ini kemudian direspons oleh Sistem melalui aktivitas "Menganalisis Dokumen", yaitu proses inti sistem dalam mendeteksi keaslian atau validitas daftar pustaka pada dokumen yang telah diunggah. Setelah proses analisis dijalankan, alur diagram melewati sebuah bar sinkronisasi sebelum akhirnya berakhir pada *final node*, yang menandakan bahwa proses upload dan permulaan analisis dokumen telah selesai dilakukan.

3.3.5. Activity Mengecek Dokumen



Gambar 3.3.5. Activity Mengecek Dokumen

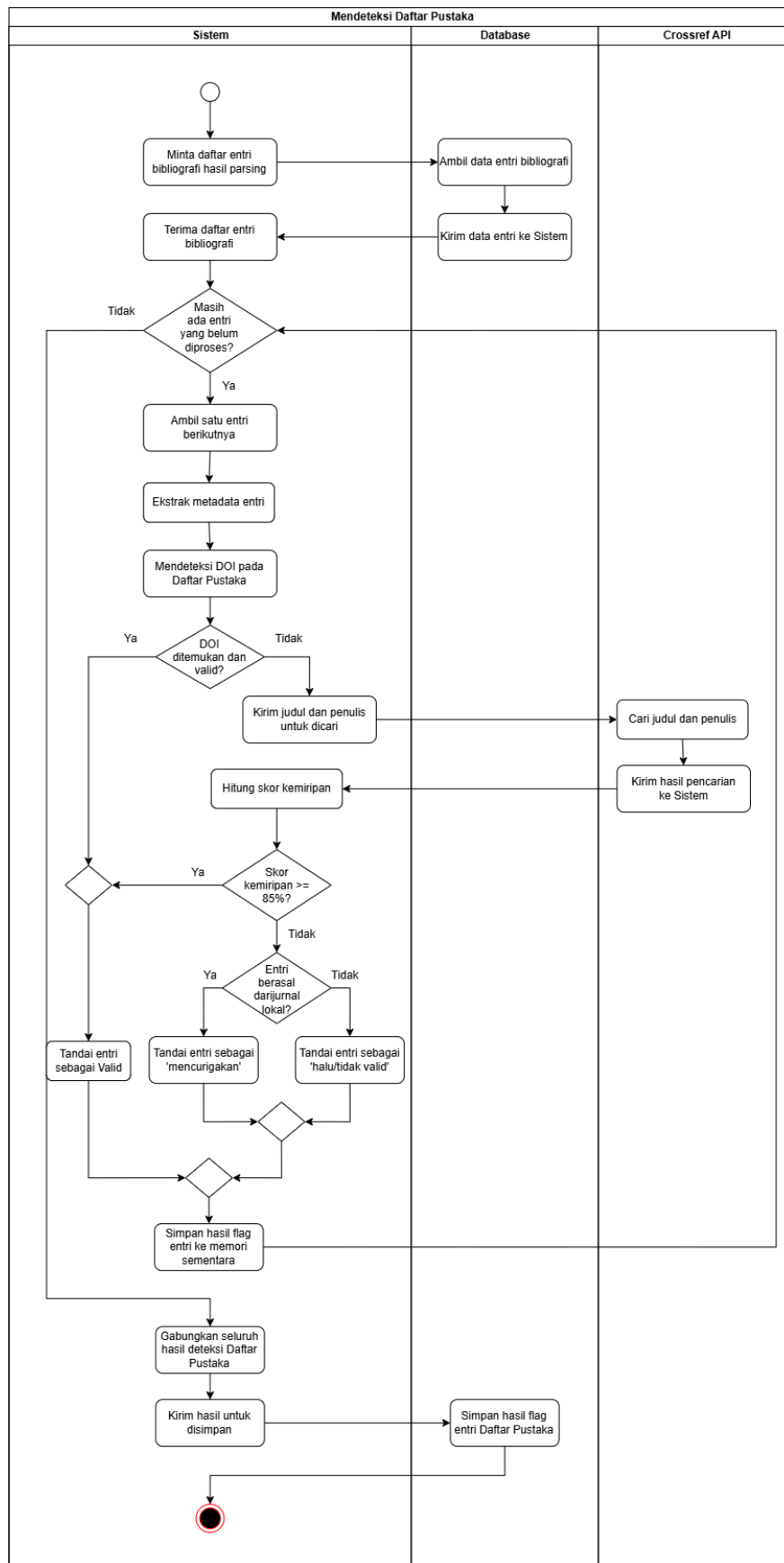
Diagram ini menjelaskan proses ketika sistem melakukan pengecekan terhadap dokumen yang sudah diupload oleh pengguna, yang melibatkan empat aktor yaitu User, Sistem, *Database*, dan *File Storage*.

Proses ini dimulai saat pengguna menekan tombol "Cek Dokumen", kemudian sistem akan meminta metadata dokumen kepada *database*, termasuk *path* atau lokasi file tersebut disimpan. Setelah mendapatkan *path*-nya, sistem kemudian meminta file dokumen tersebut ke *File Storage*, dan setelah file diterima, sistem akan melakukan *parsing* untuk mengekstrak seluruh teks dokumen sekaligus memisahkan bagian daftar pustakanya.

Pada tahap ini terdapat kondisi, apabila *parsing* gagal dilakukan maka sistem akan langsung mengubah status dokumen menjadi "gagal diproses" pada *database* dan proses berhenti sampai di situ. Tetapi apabila *parsing* berhasil, maka proses akan dilanjutkan dengan mengekstrak entri-entri *bibliography* yang ada, kemudian sistem akan mendeteksi entri mana saja yang tidak valid atau terindikasi halu, dan hasil deteksi ini kemudian digabungkan. Selain itu sistem juga mengekstrak sitasi yang terdapat pada isi teks, lalu mendeteksi sitasi mana yang tidak valid atau halu, dan hasilnya juga digabungkan.

Setelah kedua hasil deteksi tadi yaitu daftar pustaka dan sitasi sudah didapatkan, sistem akan menggabungkan seluruh hasil pengecekan dokumen tersebut menjadi satu kesatuan, lalu mengirimkannya untuk disimpan. Pada tahap ini *database* akan menyimpan hasil pengecekan sekaligus mengubah status dokumen menjadi "selesai". Aktivitas ini diakhiri dengan sistem menampilkan notifikasi hasil pengecekan kepada pengguna, sehingga pengguna menerima notifikasi tersebut sebagai tanda bahwa proses pengecekan dokumen telah selesai dilakukan.

3.3.6. Activity Mendeteksi Daftar Pustaka



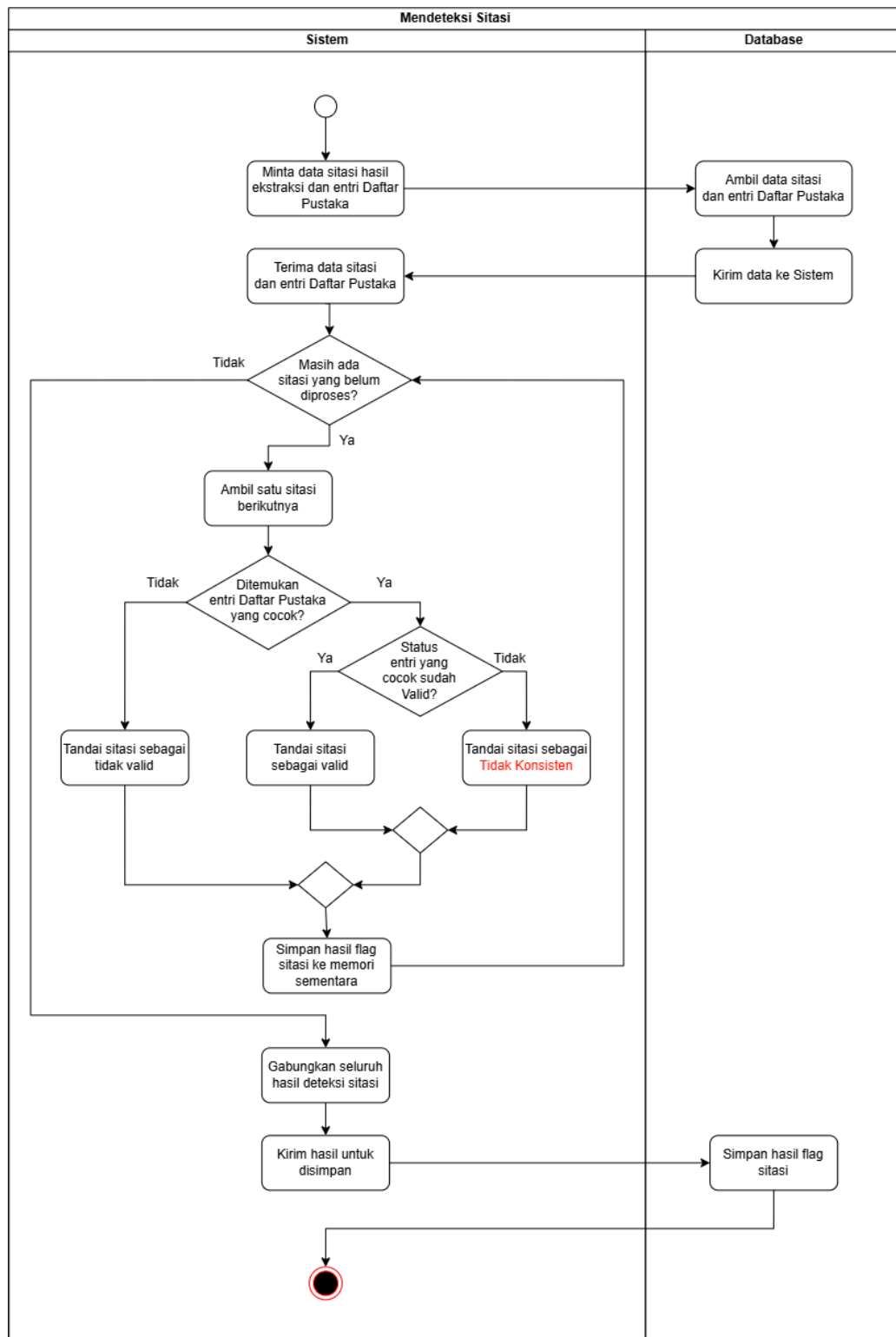
Gambar 3.3.6. Activity Mendeteksi Daftar Pustaka

Diagram ini menjelaskan proses sistem dalam mendeteksi daftar pustaka, yang melibatkan Sistem, *Database*, dan *Crossref API*. Prosesnya diawali dengan sistem meminta daftar entri bibliografi dari *database*, lalu memprosesnya satu per satu sampai seluruh entri habis. Setiap entri akan dicek terlebih dahulu apakah memiliki DOI yang valid. Jika iya, entri langsung ditandai "Valid". Jika tidak, sistem akan mengirim judul dan nama penulis ke *Crossref API* untuk dicari kecocokannya, lalu dihitung skor kemiripannya.

Apabila skor kemiripan mencapai 85 persen atau lebih, entri ditandai "Valid". Tapi kalau skornya kurang dari itu, sistem tidak langsung menandai entri tersebut sebagai *halu*, melainkan mengecek dulu apakah entri berasal dari jurnal lokal atau bukan. Pengecekan ini perlu dilakukan karena *Crossref API* sendiri lebih banyak mengindeks jurnal internasional, sedangkan banyak jurnal lokal atau jurnal Indonesia yang belum terdaftar di dalamnya. Jadi kalau ada entri dari jurnal lokal yang skor kemiripannya rendah, itu belum tentu berarti entrinya benar benar palsu, bisa saja jurnalnya memang asli tapi cuma belum terindeks di *Crossref*. Oleh karena itu entri semacam ini ditandai sebagai "mencurigakan" saja, tidak langsung divonis tidak valid, supaya tidak terjadi *false positive*. Sedangkan kalau entri tersebut bukan berasal dari jurnal lokal dan skor kemiripannya tetap rendah, maka kemungkinan besar entrinya memang tidak ditemukan sumbernya, sehingga ditandai sebagai "*halu/tidak valid*".

Hasil penandaan tiap entri disimpan sementara sampai semua entri selesai diproses. Setelah itu, seluruh hasil deteksi digabungkan dan dikirim untuk disimpan, lalu *database* menyimpan hasil *flag* daftar pustaka tersebut sebagai tahap akhir dari proses ini.

3.3.7. Activity Mendeteksi Sitasi



Gambar 3.3.7. Activity Mendeteksi Sitasi

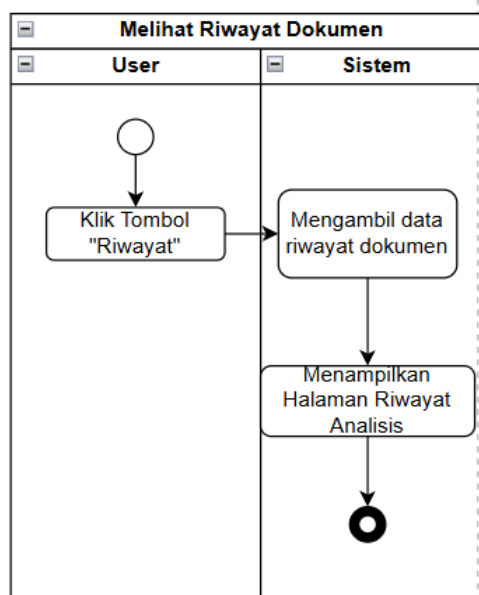
Diagram ini menjelaskan proses sistem dalam mendeteksi sitasi pada dokumen, yang melibatkan Sistem dan *Database*. Prosesnya diawali dengan sistem meminta data sitasi hasil ekstraksi beserta entri daftar pustaka yang sudah dideteksi

sebelumnya dari *database*, lalu memprosesnya satu per satu sampai seluruh sitasi habis diproses.

Untuk setiap sitasi yang diambil, sistem akan mengecek apakah sitasi tersebut memiliki entri daftar pustaka yang cocok atau tidak. Kalau ternyata tidak ditemukan entri yang cocok sama sekali, artinya sitasi tersebut merujuk ke sumber yang tidak tercantum di daftar pustaka, sehingga sitasi langsung ditandai sebagai "tidak valid". Namun kalau ditemukan entri yang cocok, sistem tidak langsung menandainya valid, tapi mengecek dulu status entri daftar pustaka yang bersangkutan, apakah sebelumnya sudah ditandai "Valid" pada proses deteksi daftar pustaka atau belum. Kalau status entrinya memang sudah valid, berarti sitasi tersebut merujuk ke sumber yang jelas keberadaannya, sehingga sitasi ikut ditandai "valid". Tapi kalau status entrinya ternyata belum valid, misalnya entrinya sendiri sempat ditandai "mencurigakan" atau "halu/tidak valid" sebelumnya, maka sitasi tersebut ditandai sebagai "Tidak Konsisten". Artinya di sini, sitasi secara struktur memang menunjuk ke entri yang ada, tapi entri yang ditunjuknya itu sendiri bermasalah, jadi keabsahan sitasinya jadi ikut diragukan.

Hasil penandaan dari tiap sitasi ini disimpan sementara ke memori, dan proses ini terus berulang sampai seluruh sitasi selesai dicek satu per satu. Setelah semua selesai, seluruh hasil deteksi sitasi tadi digabungkan menjadi satu, lalu dikirim untuk disimpan, dan *database* pun menyimpan hasil *flag* sitasi tersebut sebagai tahap akhir dari proses ini.

3.3.8. Activity Melihat Hasil Pengecekan Dokumen

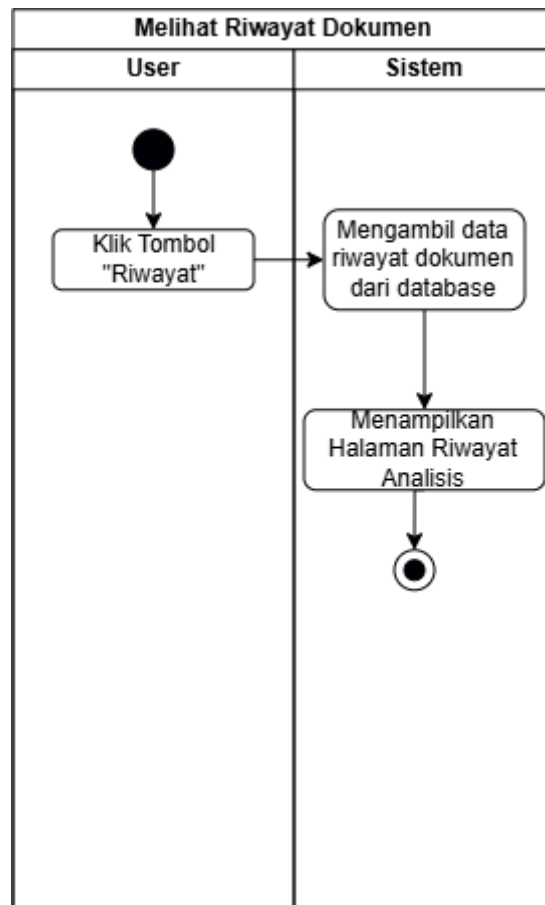


Gambar 3.3.8. Activity Melihat Hasil Pengecekan Dokumen

Activity diagram melihat hasil pengecekan dokumen menggambarkan alur aktivitas pengguna dalam melihat hasil pengecekan dokumen yang telah dilakukan sebelumnya. Proses dimulai ketika pengguna memilih tombol

“Riwayat” pada sistem. Selanjutnya, sistem menampilkan halaman riwayat analisis yang berisi dokumen-dokumen yang telah dilakukan pengecekan. Pengguna kemudian memilih salah satu dokumen untuk melihat hasil pengecekan secara lebih detail. Setelah dokumen dipilih, sistem menampilkan detail hasil pengecekan dokumen kepada pengguna. Proses berakhir setelah hasil pengecekan berhasil ditampilkan.

3.3.9. *Activity* Melihat Riwayat Dokumen

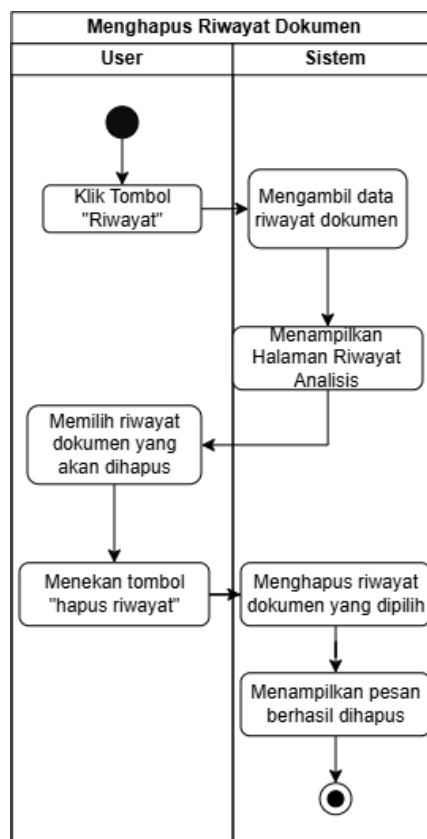


Gambar 3.3.9. *Activity* Melihat Riwayat Dokumen

Diagram aktivitas ini menggambarkan alur proses "Melihat Riwayat Dokumen" pada sistem deteksi daftar pustaka baru, yang melibatkan interaksi antara dua aktor, yaitu *User* (pengguna) dan Sistem. Proses dimulai dari sisi *User*, di mana pengguna melakukan aksi "Klik Tombol Riwayat" untuk meminta sistem menampilkan riwayat dokumen yang sebelumnya pernah diperiksa keaslian daftar pustakanya. Setelah tombol tersebut di klik, kendali proses berpindah ke sisi Sistem, yang kemudian menjalankan aktivitas "Mengambil data riwayat dokumen dari *database*", yaitu proses pengambilan data hasil-hasil pemeriksaan atau deteksi daftar pustaka baru yang telah tersimpan sebelumnya, seperti daftar dokumen yang pernah dianalisis beserta status atau hasil deteksinya. Setelah data berhasil diambil, sistem melanjutkan ke aktivitas berikutnya yaitu "Menampilkan

Halaman Riwayat Analisis", di mana sistem menyajikan informasi riwayat tersebut kepada pengguna dalam bentuk halaman yang dapat dilihat, misalnya berisi daftar dokumen yang pernah diunggah beserta ringkasan hasil deteksi apakah daftar pustakanya terindikasi halu (tidak valid/fiktif) atau valid. Alur proses ini kemudian diakhiri pada sisi Sistem, yang menandakan bahwa proses menampilkan riwayat dokumen telah selesai dilakukan. Secara keseluruhan, diagram ini menunjukkan bahwa peran *User* hanya sebatas memicu permintaan melalui satu aksi klik, sedangkan seluruh proses pengambilan dan penyajian data riwayat deteksi sepenuhnya ditangani oleh Sistem secara otomatis.

3.3.10. *Activity Menghapus Riwayat Dokumen*



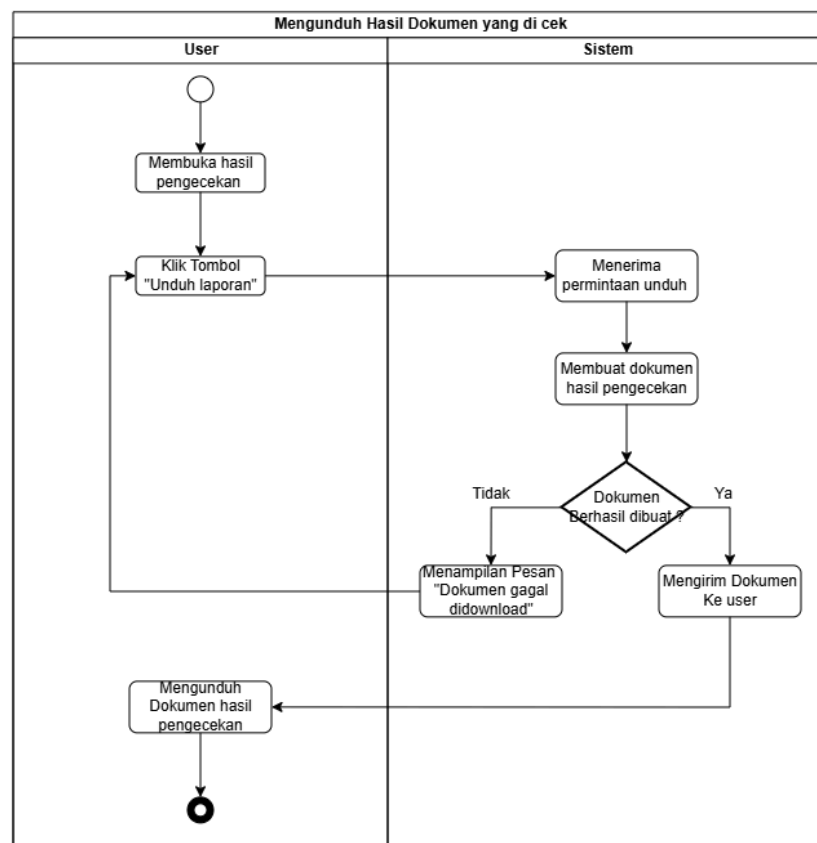
Gambar 3.3.10. *Activity Menghapus Riwayat Dokumen*

Diagram aktivitas ini menggambarkan alur proses "Menghapus Riwayat Dokumen" pada sistem deteksi daftar pustaka halu, yang juga melibatkan interaksi antara *User* dan Sistem secara bergantian. Proses dimulai dari sisi *User*, di mana pengguna melakukan aksi "Klik Tombol Riwayat" untuk membuka daftar riwayat dokumen yang pernah diperiksa. Setelah tombol diklik, kendali berpindah ke sisi Sistem yang menjalankan aktivitas "Mengambil data riwayat dokumen", yaitu proses pengambilan seluruh data riwayat hasil deteksi daftar pustaka yang tersimpan di database, kemudian dilanjutkan dengan aktivitas "Menampilkan Halaman Riwayat Analisis" untuk menyajikan data tersebut kepada pengguna.

dalam bentuk halaman yang dapat dilihat dan dipilih. Setelah halaman riwayat ditampilkan, kendali kembali ke sisi *User*, di mana pengguna melakukan aktivitas "Memilih riwayat dokumen yang akan dihapus", yaitu menentukan dokumen mana dari daftar riwayat yang ingin dihapus datanya.

Setelah dokumen yang dimaksud dipilih, pengguna melanjutkan dengan aksi "Menekan tombol hapus riwayat" untuk mengonfirmasi permintaan penghapusan. Permintaan tersebut kemudian diproses oleh Sistem melalui aktivitas "Menghapus riwayat dokumen yang dipilih", yaitu proses penghapusan data riwayat analisis tertentu dari database sesuai dengan pilihan pengguna, dan setelah proses penghapusan berhasil dilakukan, sistem menjalankan aktivitas "Menampilkan pesan berhasil dihapus" sebagai bentuk konfirmasi kepada pengguna bahwa data riwayat tersebut telah berhasil dihapus.

3.3.11. *Activity* Mengunduh Hasil Dokumen yang di-cek

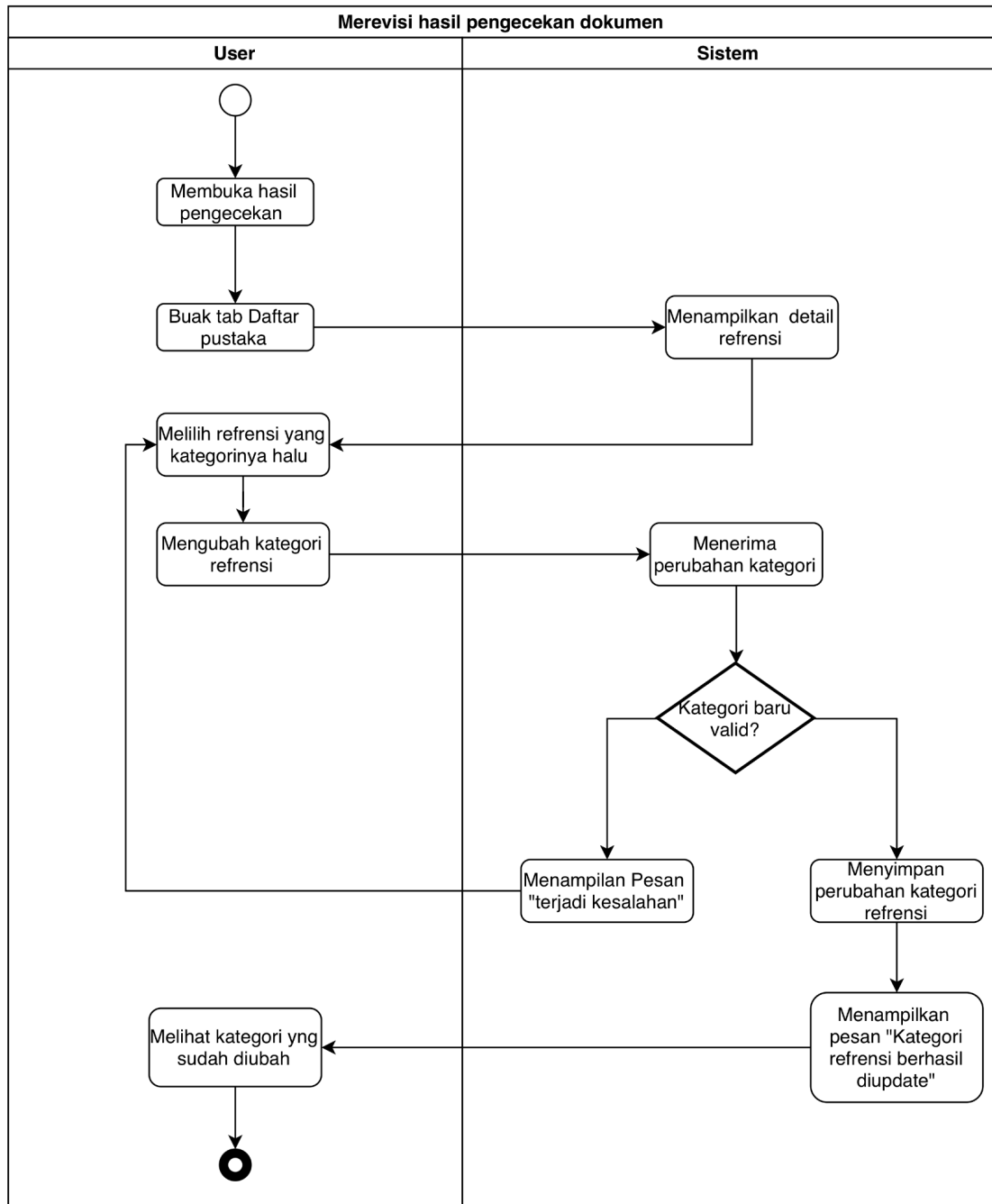


Gambar 3.3.11. *Activity* Mengunduh Hasil Dokumen yang di-cek

Activity diagram mengunduh hasil dokumen yang dicek menggambarkan proses pengguna dalam mengunduh dokumen hasil pengecekan. Proses dimulai ketika pengguna membuka hasil pengecekan dan memilih tombol "Unduh Laporan". Selanjutnya, sistem menerima permintaan unduh dan membuat dokumen hasil pengecekan. Sistem kemudian melakukan pengecekan apakah dokumen berhasil dibuat. Jika berhasil, dokumen hasil pengecekan dikirimkan kepada pengguna dan

pengguna dapat mengunduh dokumen tersebut. Jika dokumen gagal dibuat, sistem menampilkan pesan “Dokumen gagal diunduh” kepada pengguna.

3.3.12. *Activity* Merevisi Hasil Pengecekan Dokumen



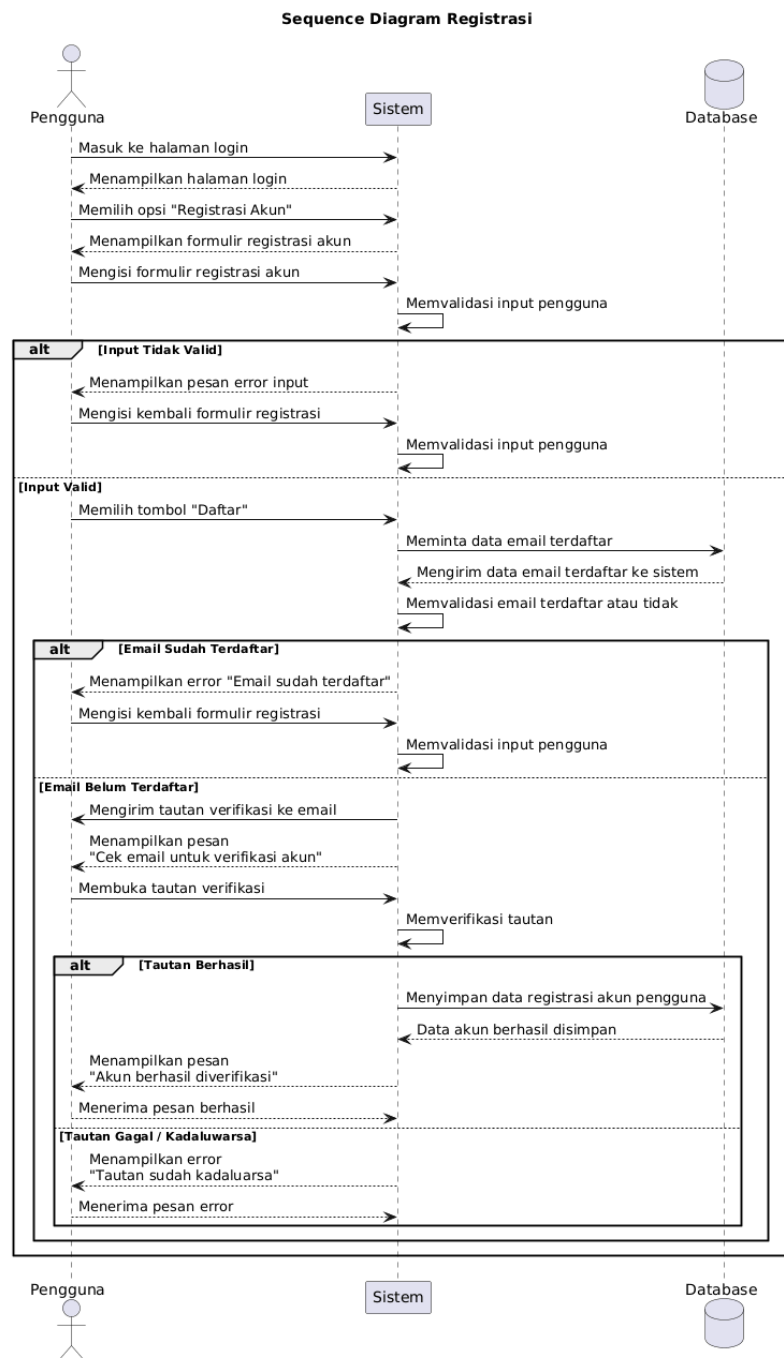
Gambar 3.3.12. *Activity* Merevisi Hasil Pengecekan Dokumen

Seperti yang digambarkan dalam Gambar 3.3.12, merevisi hasil pengecekan dokumen diawali dengan *User* yang membuka hasil pengecekan dan membuka tab daftar pustaka. Sistem kemudian merespons dengan menampilkan daftar-daftar referensi. Pengguna dapat memilah dan memilih referensi mana saja yang

terindikasi *false flag* oleh sistem dan memberikan koreksi kategori yang benar. Sistem kemudian menyimpan perubahan kategori referensi yang dipilih dan menampilkan pesan bahwa kategori referensi berhasil diperbaharui. Jika kategori yang dipilih tidak valid, sistem akan mengembalikan pesan error bahwa terjadi kesalahan.

3.4 Sequence Diagram

3.4.1. Register

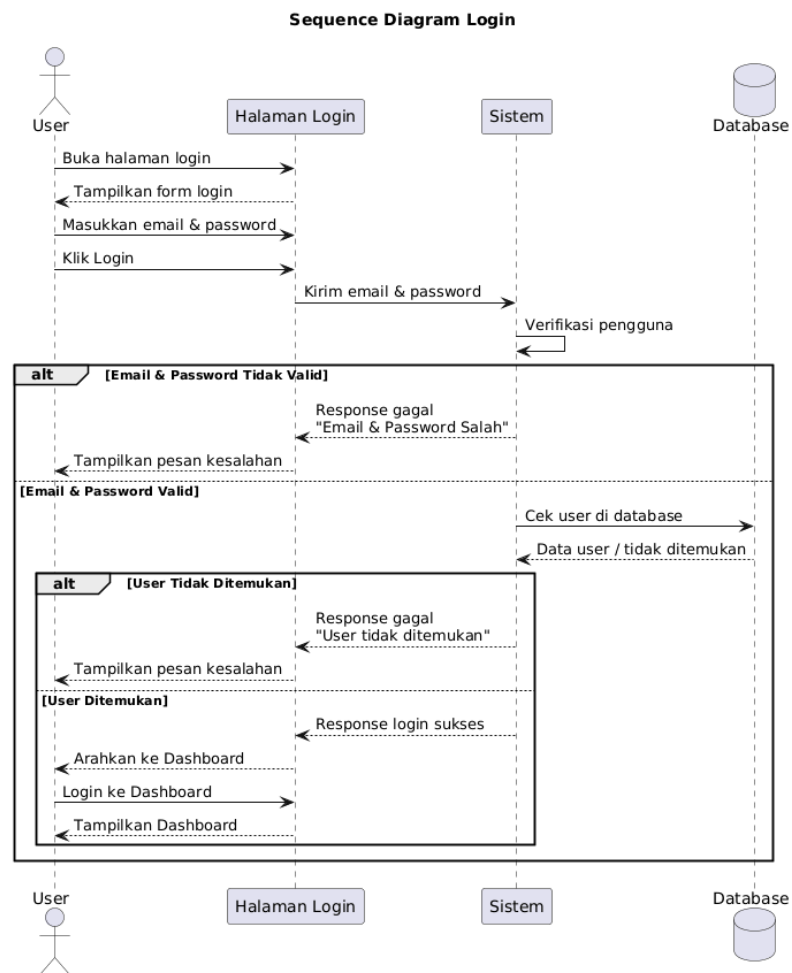


Gambar 3.3.1. Sequence Register

Proses registrasi dimulai saat pengguna masuk ke halaman login, kemudian memilih opsi "Registrasi Akun" sehingga sistem menampilkan formulir registrasi. Pengguna mengisi formulir tersebut, dan Sistem akan memvalidasi input yang dimasukkan. Apabila input yang diisi tidak valid, Sistem akan menampilkan pesan *error* dan meminta pengguna mengisi ulang formulir hingga validasinya berhasil.

Setelah input dinyatakan valid, pengguna menekan tombol "Daftar", lalu Sistem meminta data email yang sudah terdaftar dari *Database* untuk memastikan email tersebut belum pernah digunakan sebelumnya. Jika ternyata email sudah terdaftar, Sistem akan menampilkan pesan *error* dan meminta pengguna mengisi ulang formulirnya. Namun jika email belum terdaftar, Sistem akan mengirimkan tautan verifikasi ke email pengguna dan menampilkan pesan agar pengguna mengecek emailnya. Setelah pengguna membuka tautan verifikasi tersebut, Sistem akan memeriksa keabsahan tautan itu. Apabila tautan terbukti valid, Sistem akan menyimpan data registrasi ke *Database* dan menampilkan pesan bahwa akun berhasil diverifikasi. Sebaliknya, jika tautan sudah tidak berlaku atau kadaluarsa, Sistem akan menampilkan pesan *error* yang menandakan bahwa proses verifikasi tersebut gagal dilakukan.

3.4.2. Sequence Login

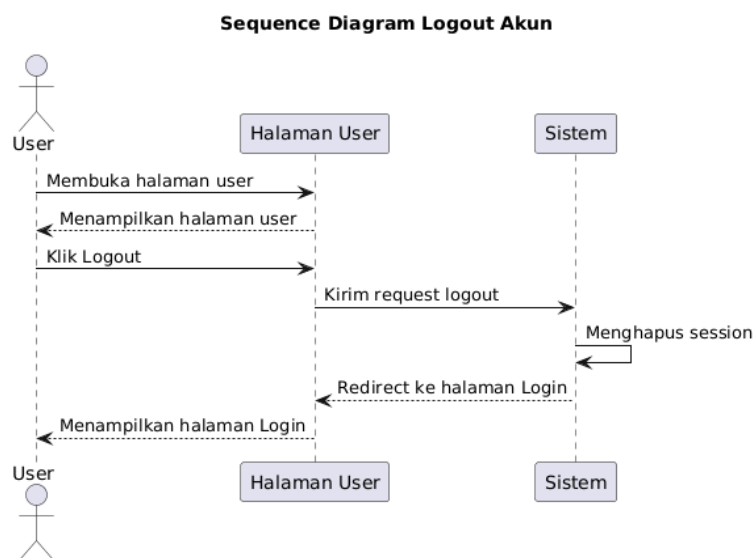


Gambar 3.3.2. Sequence Login

Proses *login* dimulai ketika pengguna membuka halaman *login*, lalu sistem menampilkan *form login* untuk diisi. Pengguna kemudian memasukkan email dan password, lalu menekan tombol *login*. Data yang dimasukkan tadi dikirim oleh

Halaman *Login* ke Sistem, dan Sistem akan melakukan verifikasi terhadap data tersebut. Jika email dan password yang dimasukkan tidak valid, Sistem akan mengirim respons gagal berupa pesan "Email & Password Salah" yang kemudian ditampilkan kepada pengguna. Namun jika email dan password valid, Sistem akan melanjutkan proses dengan mengecek keberadaan user tersebut di *Database*. Apabila user tidak ditemukan di *Database*, Sistem akan menampilkan pesan kesalahan bahwa user tidak ditemukan. Sebaliknya, jika user ditemukan, Sistem akan mengirim *respons login* sukses dan mengarahkan pengguna menuju *Dashboard*, sehingga tampilan *Dashboard* pun berhasil ditampilkan kepada pengguna.

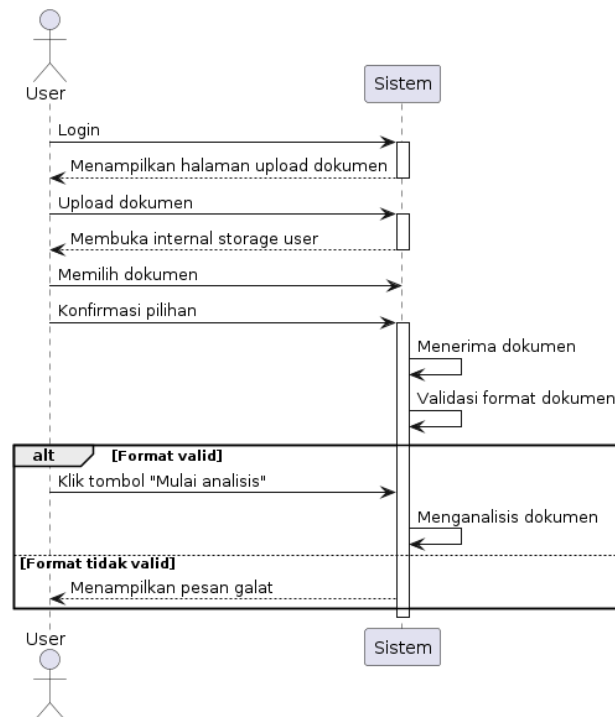
3.4.3. Sequence Logout



Gambar 3.3.3. Sequence Logout

Proses logout diawali dengan pengguna membuka halaman user, yang kemudian ditampilkan oleh sistem. Setelah berada di halaman tersebut, pengguna menekan tombol *logout*, dan permintaan logout ini diteruskan oleh Halaman User ke Sistem. Sistem kemudian melakukan proses internal untuk menghapus session milik pengguna, sebagai tanda bahwa sesi *login* pengguna sudah tidak berlaku lagi. Setelah *session* dihapus, Sistem akan mengarahkan pengguna kembali ke halaman *login* melalui proses *redirect*. Terakhir, halaman *login* pun ditampilkan kepada pengguna sebagai tanda bahwa proses *logout* telah selesai dilakukan.

3.4.4. Sequence Mengunggah Dokumen

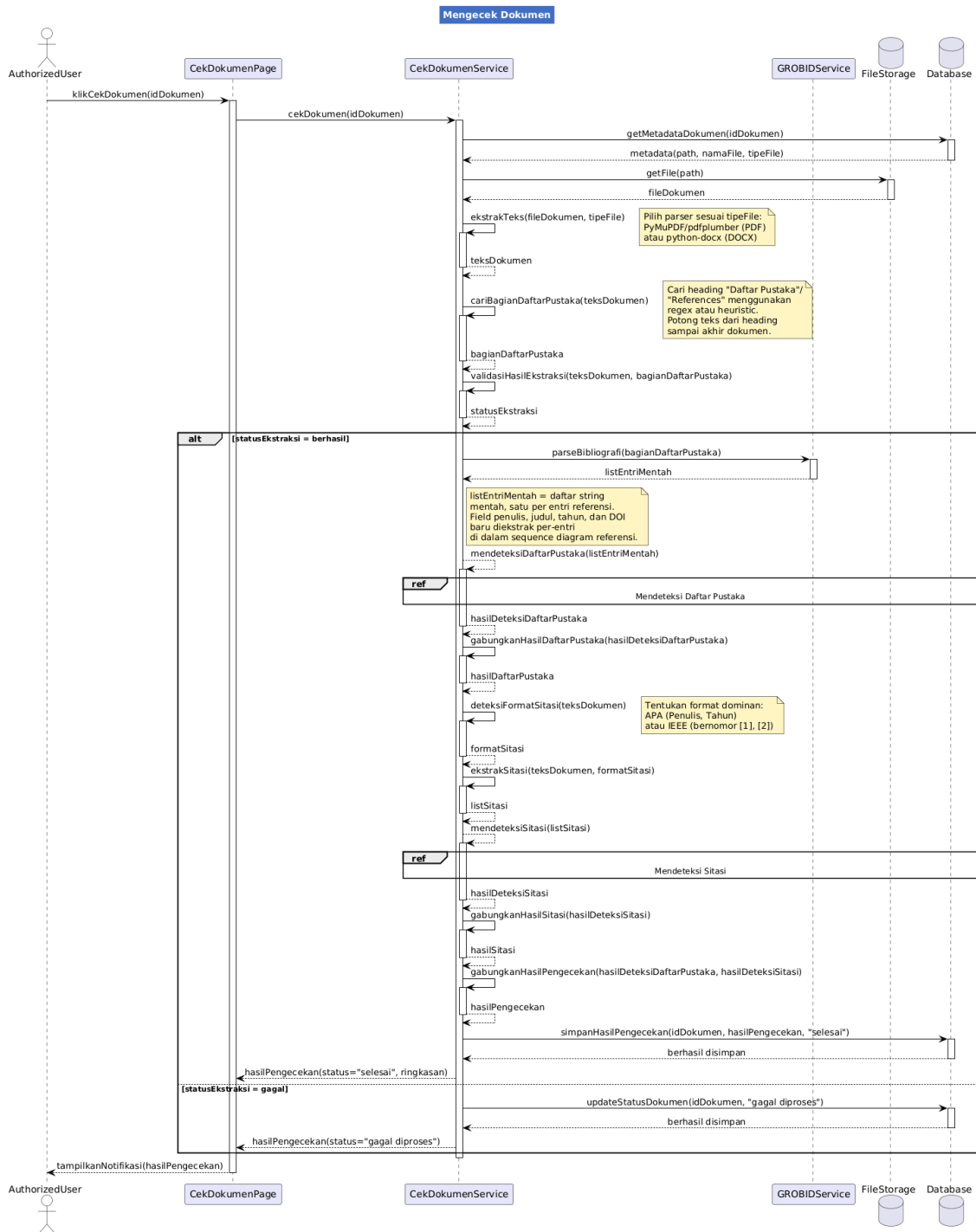


Gambar 3.3.4. Sequence Mengunggah Dokumen

Proses diawali dengan pengguna melakukan *login* ke dalam sistem. Setelah proses autentikasi berhasil, sistem akan menampilkan halaman upload dokumen kepada pengguna. Selanjutnya, pengguna mengunggah dokumen yang dimaksud, kemudian sistem membuka akses ke penyimpanan internal milik pengguna guna memfasilitasi proses pemilihan berkas. Pengguna kemudian memilih dokumen yang akan diunggah dan mengonfirmasi pilihan tersebut. Atas dasar konfirmasi tersebut, sistem menerima dokumen yang diunggah dan melakukan proses validasi terhadap format dokumen tersebut.

Apabila hasil validasi menunjukkan bahwa format dokumen sesuai dengan ketentuan yang berlaku, maka pengguna dapat melanjutkan proses dengan menekan tombol "Mulai Analisis", yang kemudian akan memicu sistem untuk melakukan analisis terhadap dokumen guna mendeteksi indikasi daftar pustaka yang tidak valid atau fiktif. Sebaliknya, apabila format dokumen dinyatakan tidak sesuai, sistem akan menampilkan pesan kesalahan (pesan galat) kepada pengguna sebagai pemberitahuan bahwa dokumen tersebut tidak dapat diproses lebih lanjut, sehingga pengguna diharuskan untuk mengunggah kembali dokumen dengan format yang sesuai.

3.4.5. Sequence Mengecek Dokumen



Gambar 3.4.5. Sequence Mengecek Dokumen

Diagram ini menjelaskan interaksi antar objek saat proses pengecekan dokumen berlangsung, yang melibatkan *AuthorizedUser*, *CekDokumenPage*, *CekDokumenService*, *GROBIDService*, *FileStorage*, dan *Database*. Prosesnya dimulai ketika *AuthorizedUser* menekan tombol cek dokumen, lalu *CekDokumenPage* meneruskan permintaan ini ke *CekDokumenService*. Dari sini sistem meminta metadata dokumen ke *Database*, berupa *path*, nama file, dan tipe

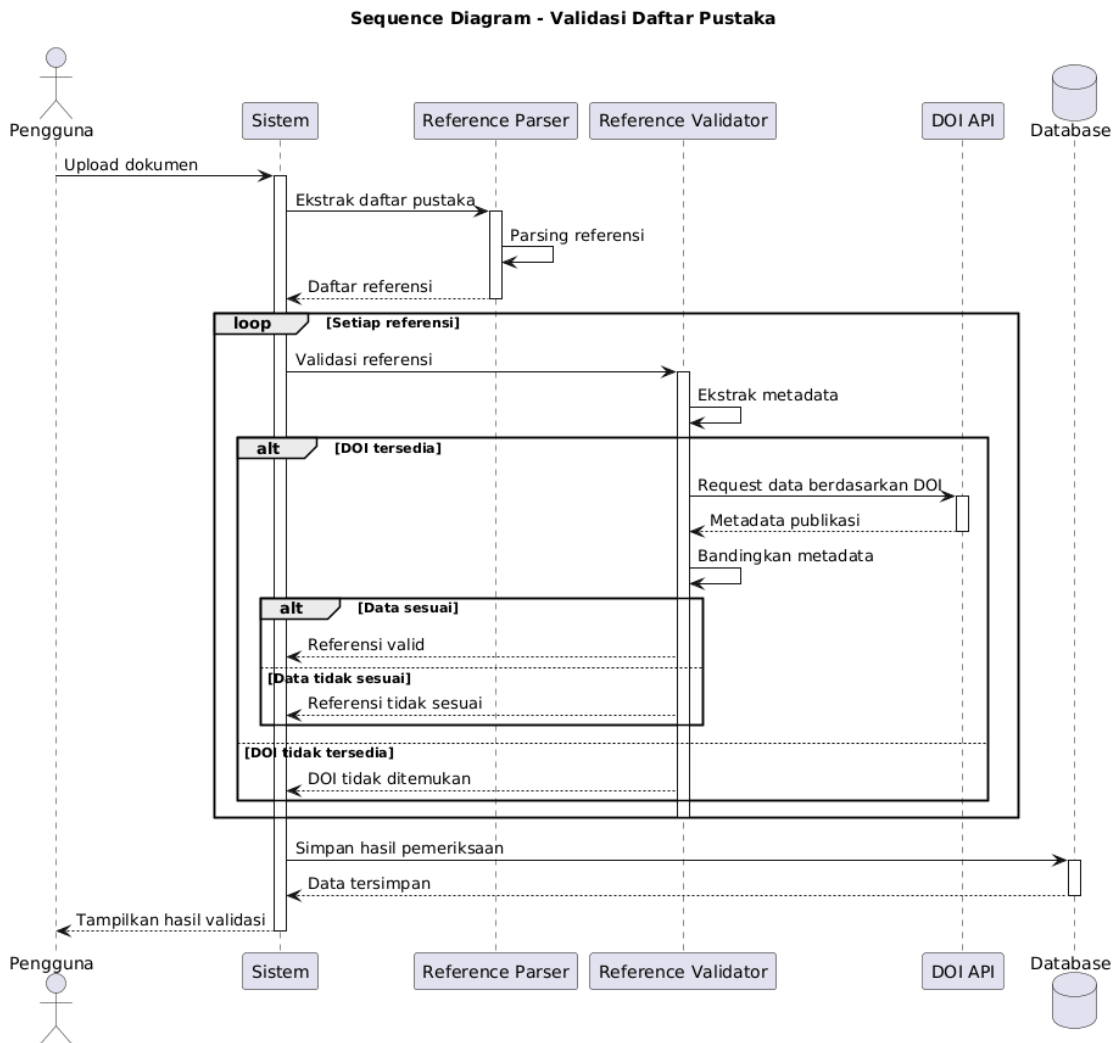
filenya, kemudian berdasarkan *path* tersebut sistem mengambil file dokumennya langsung dari *FileStorage*.

Setelah file diterima, sistem mengekstrak teks dari dokumen tersebut sesuai tipe filenya, lalu mencari bagian daftar pustaka di dalam teks tersebut. Hasil dari kedua ekstraksi ini kemudian divalidasi untuk menentukan status ekstraksinya, apakah berhasil atau gagal, karena ini yang akan menentukan alur mana yang dijalankan selanjutnya.

Kalau ekstraksinya berhasil, bagian daftar pustaka yang sudah ditemukan tadi dikirim ke *GROBIDService* untuk diparsing menjadi daftar entri mentah per referensi. Daftar entri ini kemudian diproses lebih lanjut lewat *sub-process* "Mendeteksi Daftar Pustaka", dimana setiap entrinya di cek satu per satu, dimulai dari DOI-nya, lalu kalau tidak ditemukan baru dicocokkan lewat *Crossref API* untuk dilihat skor kemiripannya, sampai akhirnya setiap entri mendapat status Valid, mencurigakan, atau halu/tidak valid. Bersamaan dengan itu, sistem juga menentukan format sitasi yang dipakai dokumen, mengekstrak seluruh sitasinya, lalu memprosesnya lewat *sub-process* "Mendeteksi Sitasi" yang mencocokkan tiap sitasi dengan entri daftar pustaka yang sudah di cek statusnya tadi.

Hasil dari kedua deteksi ini, yaitu hasil daftar pustaka dan hasil sitasi, kemudian digabungkan menjadi satu hasil pengecekan dokumen secara keseluruhan dan disimpan ke *Database* dengan status "selesai". Sebaliknya, kalau ekstraksinya gagal di awal, alurnya jauh lebih singkat, *CekDokumenService* langsung mengubah status dokumen di *Database* menjadi "gagal diproses" tanpa perlu melewati tahap deteksi daftar pustaka maupun sitasi. Pada kedua kondisi ini, hasil akhirnya tetap sama sama dikembalikan dan ditampilkan sebagai notifikasi kepada *AuthorizedUser*.

3.4.6. Sequence Mendeteksi Daftar Pustaka

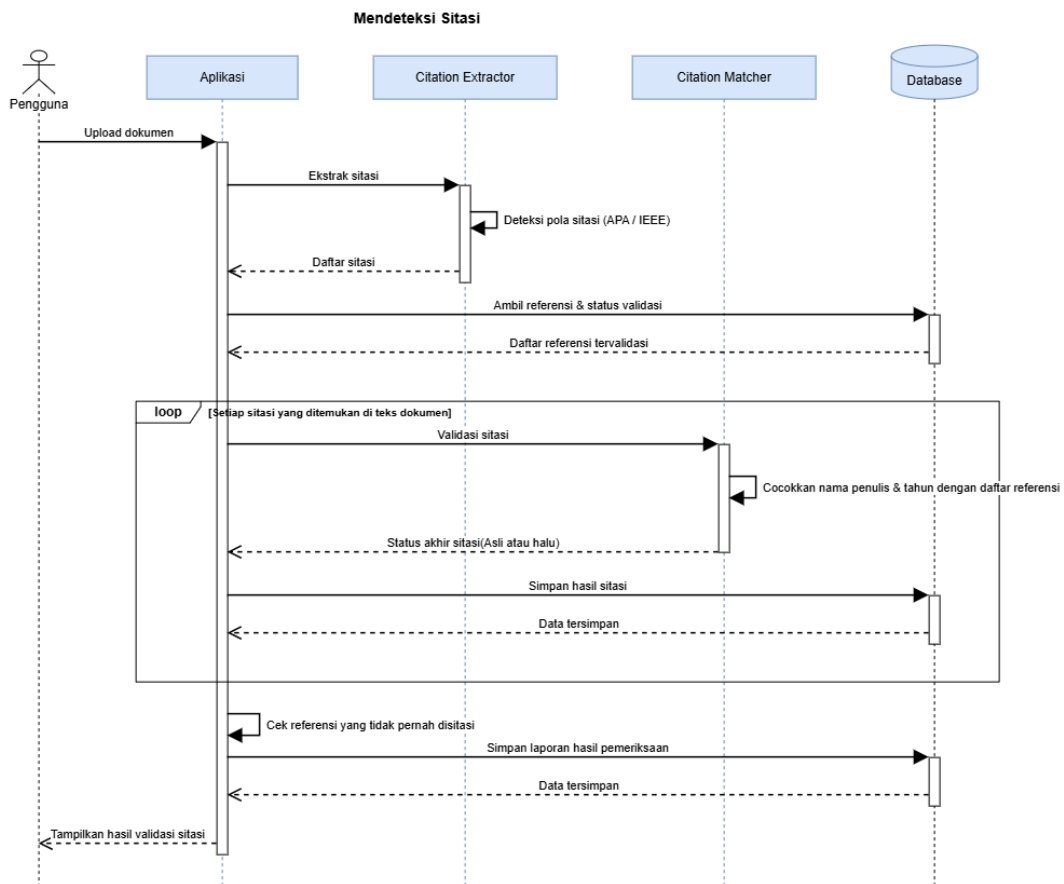


Gambar 3.4.6. Sequence Mendeteksi Daftar Pustaka

Diagram ini menggambarkan alur interaksi sistem. Proses dimulai ketika pengguna mengupload dokumen, setelah dokumen diterima sistem meneruskan dokumen kepada *parser* referensi untuk memisahkan referensi-referensi yang ada di daftar pustaka. Setelahnya setiap referensi di proses untuk mengekstrak nama penulis, judul, tahun publikasi, nama jurnal atau penerbit, serta DOI.

DOI kemudian akan dibandingkan dengan ke DOI API. DOI API yang digunakan ialah Crossref API. Hasil dari perbandingan digunakan untuk menentukan ketepatan informasi referensi dokumen dengan data yang ada pada layanan Crossref. Hasil tersebut akan disimpan di database dan ditampilkan ke pengguna.

3.4.7. Sequence Mendeteksi Sitasi



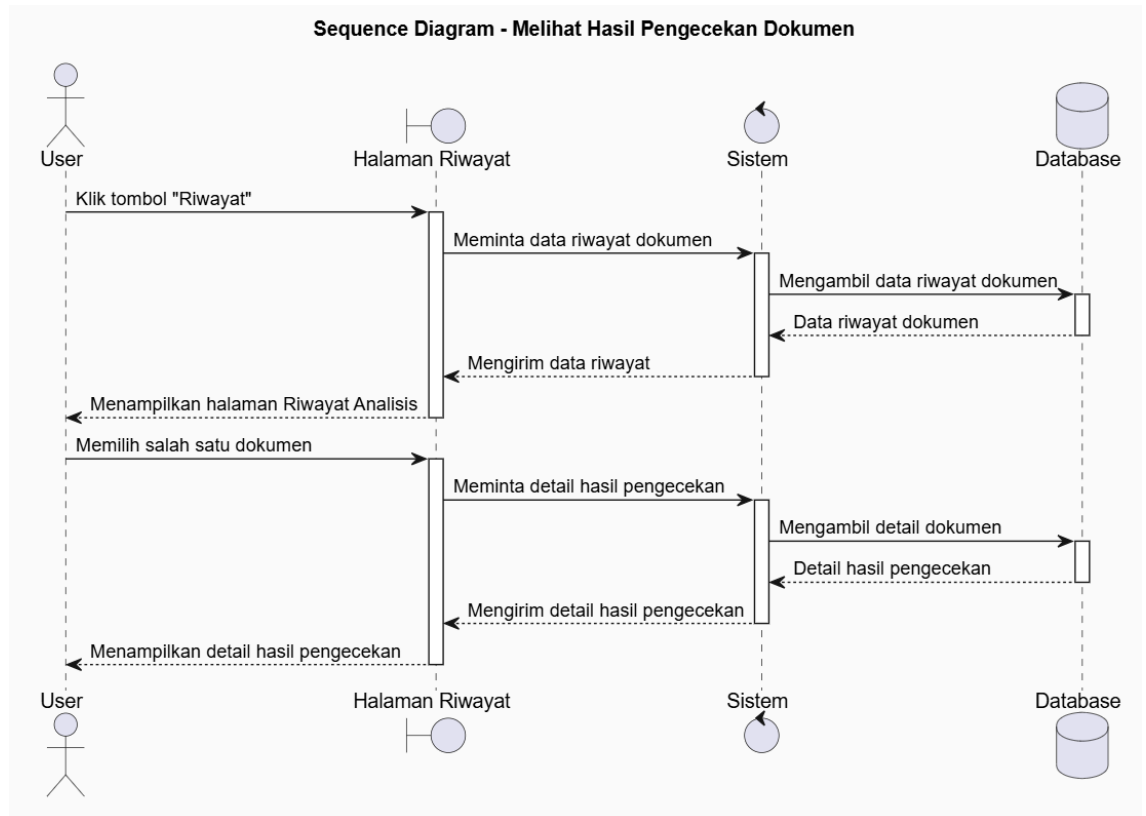
Gambar 3.4.7. Sequence Mendeteksi Sitasi

Proses deteksi sitasi dimulai ketika pengguna mengupload dokumen ke Aplikasi. Aplikasi kemudian meneruskan dokumen tersebut ke *Citation Extractor* untuk mengekstrak seluruh sitasi yang ada di dalam teks. *Citation Extractor* mengenali pola penulisan sitasi, baik gaya APA maupun IEEE, lalu mengambil nama penulis dan tahun publikasinya, dan hasil ekstraksi tersebut dikirim kembali ke Aplikasi berupa daftar sitasi. Setelah itu, Aplikasi mengambil daftar referensi beserta status validasinya dari *Database*, yaitu hasil dari proses pengecekan daftar pustaka yang sudah dilakukan sebelumnya. Selanjutnya, untuk setiap sitasi yang ditemukan, Aplikasi meminta *Citation Matcher* untuk memvalidasi sitasi tersebut. *Citation Matcher* akan mencocokkan nama penulis dan tahun pada sitasi dengan daftar referensi yang ada.

Jika sitasi tersebut cocok dengan salah satu referensi dan referensinya berstatus valid, maka *Citation Matcher* akan mengembalikan status "sitasi valid" ke Aplikasi. Jika sitasi cocok dengan suatu referensi namun referensi tersebut bermasalah atau DOI-nya tidak ditemukan, maka status yang dikembalikan adalah "sitasi halu karena referensinya tidak valid". Sebaliknya, jika sitasi tidak memiliki

referensi yang cocok sama sekali di daftar pustaka, maka *Citation Matcher* akan mengembalikan status "sitasi halu karena tidak memiliki referensi". Setiap status akhir tersebut kemudian disimpan oleh Aplikasi ke dalam *Database*. Setelah seluruh sitasi selesai diperiksa, Aplikasi turut memeriksa referensi yang ada di daftar pustaka namun tidak pernah disitasi di dalam teks dokumen. Seluruh hasil pemeriksaan tersebut kemudian disimpan sebagai laporan ke dalam *Database*, dan pada akhirnya ditampilkan kepada pengguna sebagai hasil validasi sitasi.

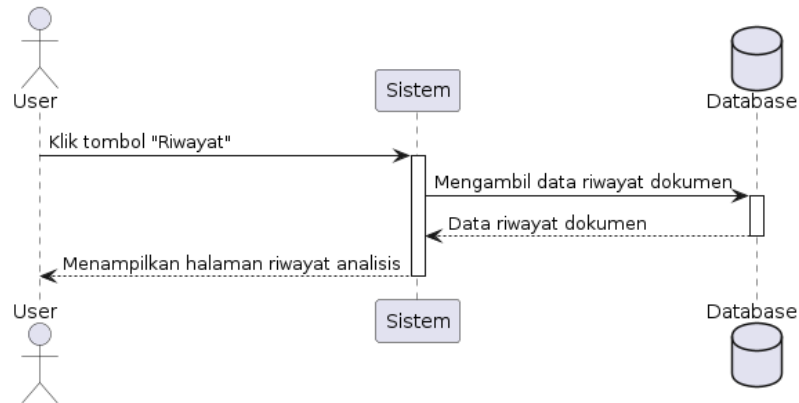
3.4.8. Sequence Melihat Hasil Pengecekan Dokumen



Gambar 3.4.8. Sequence Melihat Hasil Pengecekan Dokumen

Sequence diagram melihat hasil pengecekan dokumen menggambarkan interaksi antara pengguna, halaman riwayat, sistem, dan database dalam menampilkan hasil pengecekan dokumen. Proses dimulai ketika pengguna menekan tombol “Riwayat” pada halaman sistem. Halaman kemudian mengirimkan permintaan kepada sistem untuk mengambil data riwayat dokumen. Sistem mengambil data tersebut dari database dan mengirimkan hasilnya kembali ke halaman riwayat untuk ditampilkan kepada pengguna. Selanjutnya, pengguna memilih salah satu dokumen untuk melihat detail hasil pengecekan. Sistem kembali meminta data detail dokumen kepada database, kemudian database mengembalikan data tersebut kepada sistem. Data hasil pengecekan selanjutnya dikirimkan ke halaman dan ditampilkan kepada pengguna.

3.4.9. *Sequence* Melihat Riwayat Dokumen

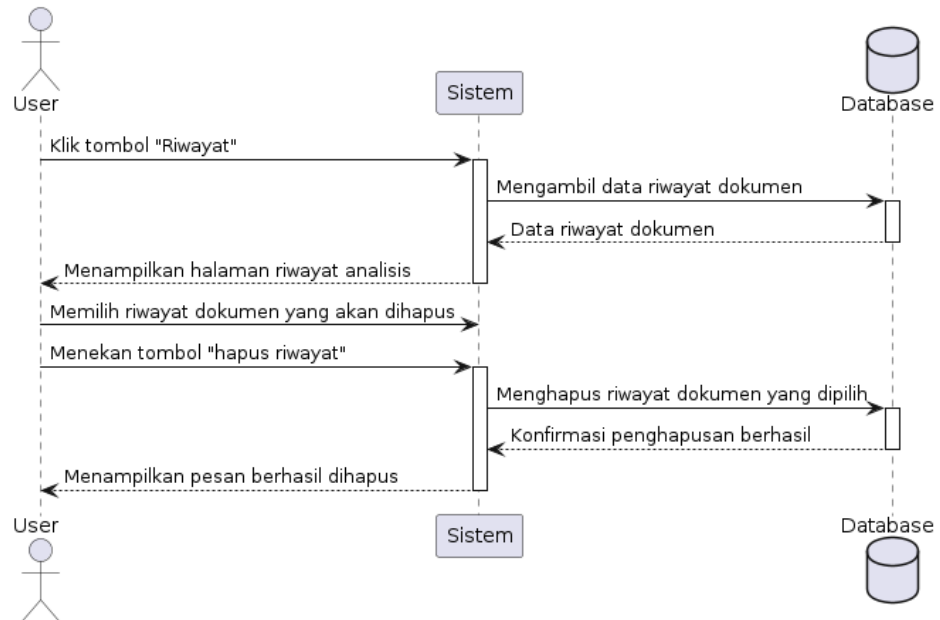


Gambar 3.4.9. *Sequence* Melihat Riwayat Dokumen

Proses ini diawali dengan tindakan pengguna yang menekan tombol "Riwayat" pada antarmuka sistem, dengan tujuan untuk memperoleh informasi mengenai riwayat dokumen yang pernah diunggah dan dianalisis sebelumnya. Atas permintaan tersebut, sistem kemudian melakukan permintaan data ke *database* dengan mengirimkan instruksi untuk mengambil seluruh data riwayat dokumen yang tersimpan. *Database*, sebagai penyimpan data utama, memproses permintaan tersebut dan mengembalikan data riwayat dokumen yang diminta kepada sistem sebagai respons atas permintaan yang diajukan.

Setelah sistem menerima data riwayat dokumen dari *database*, sistem selanjutnya memproses data tersebut untuk kemudian ditampilkan kepada pengguna dalam bentuk halaman riwayat analisis. Melalui halaman tersebut, pengguna dapat melihat secara menyeluruh riwayat dokumen yang pernah diunggah, termasuk informasi terkait hasil analisis atau status deteksi daftar pustaka pada masing-masing dokumen, apakah dinyatakan valid atau terindikasi sebagai daftar pustaka halus. Dengan demikian, keseluruhan proses ini memungkinkan pengguna untuk memperoleh gambaran historis atas aktivitas analisis dokumen yang telah dilakukan sebelumnya melalui sistem.

3.4.10. *Sequence Menghapus Riwayat Dokumen*



Gambar 3.4.10. *Sequence Menghapus Riwayat Dokumen*

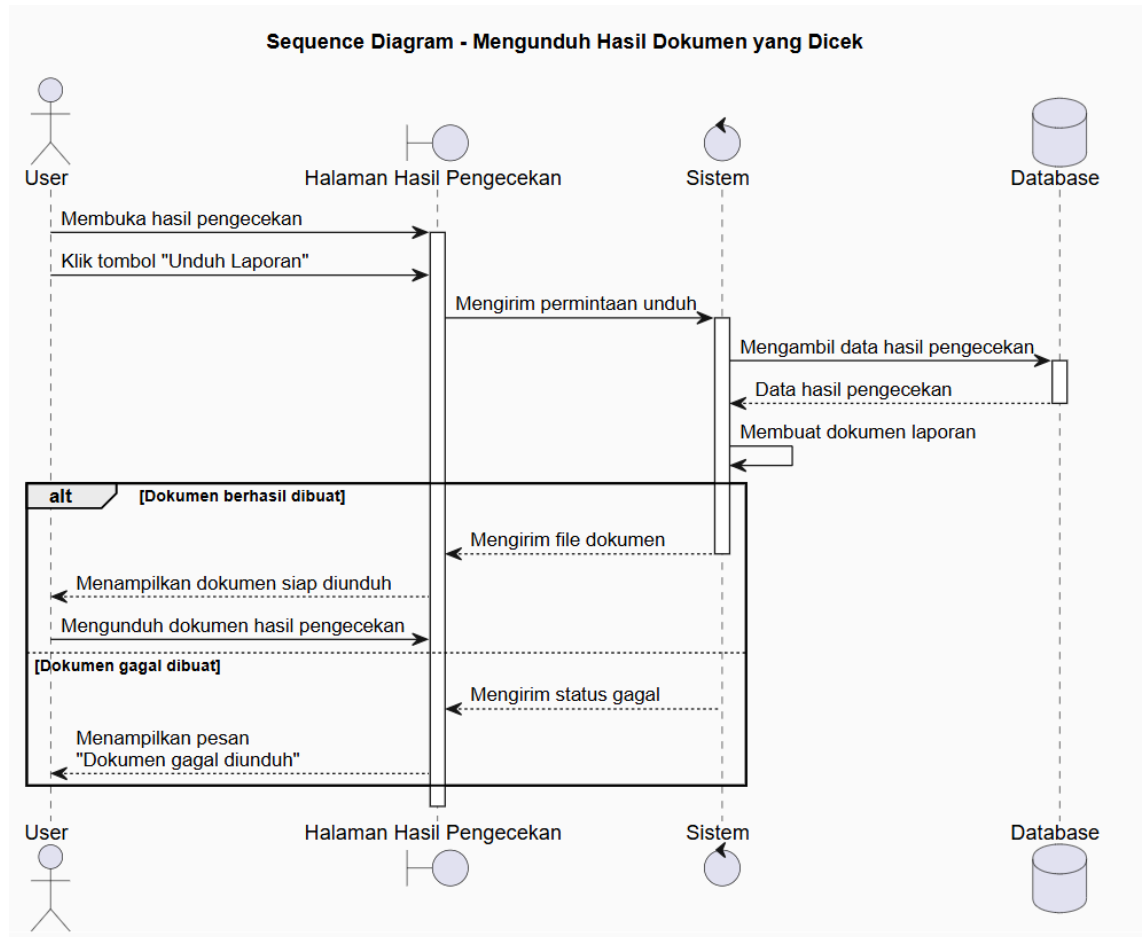
Proses ini diawali dengan tindakan pengguna yang menekan tombol "Riwayat" pada antarmuka sistem untuk memperoleh informasi mengenai riwayat dokumen yang pernah diunggah dan dianalisis sebelumnya. Atas permintaan tersebut, sistem mengirimkan instruksi kepada *database* untuk mengambil seluruh data riwayat dokumen yang tersimpan. *Database* kemudian memproses permintaan tersebut dan mengembalikan data riwayat dokumen yang diminta kepada sistem. Setelah data tersebut diterima, sistem menampilkan halaman riwayat analisis kepada pengguna, sehingga pengguna dapat melihat daftar riwayat dokumen yang pernah dianalisis beserta status hasil deteksinya.

Selanjutnya, pengguna memilih salah satu riwayat dokumen yang ingin dihapus dari daftar yang ditampilkan. Setelah menentukan dokumen yang dimaksud, pengguna menekan tombol "Hapus Riwayat" untuk mengonfirmasi permintaan penghapusan data tersebut. Permintaan ini kemudian diteruskan oleh sistem kepada *database* dengan instruksi untuk menghapus data riwayat dokumen yang telah dipilih oleh pengguna. *Database* memproses permintaan penghapusan tersebut dan selanjutnya mengirimkan konfirmasi kepada sistem bahwa proses penghapusan data telah berhasil dilakukan.

Setelah menerima konfirmasi tersebut, sistem menampilkan pesan pemberitahuan kepada pengguna bahwa riwayat dokumen yang dipilih telah

berhasil dihapus. Dengan demikian, keseluruhan proses ini memastikan bahwa penghapusan data riwayat dokumen dilakukan secara terstruktur, mulai dari pengambilan data, pemilihan dokumen yang akan dihapus, eksekusi penghapusan pada basis data, hingga pemberian umpan balik akhir kepada pengguna sebagai tanda bahwa proses telah selesai dilaksanakan.

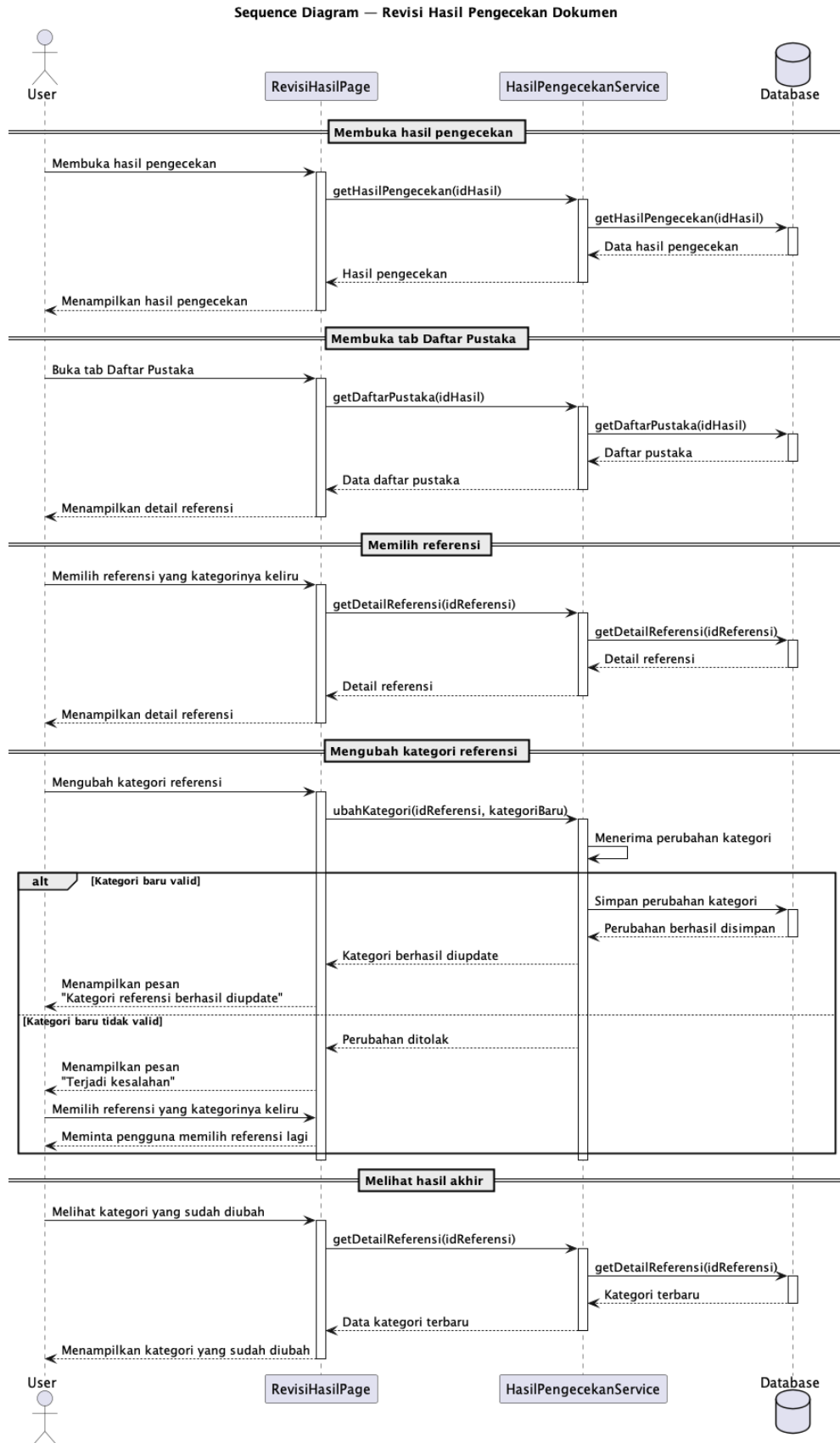
3.4.11. *Sequence Mengunduh Hasil Dokumen yang di-cek*



Gambar 3.4.11. *Sequence Mengunduh Hasil Dokumen yang di-cek*

Sequence diagram mengunduh hasil dokumen yang dicek menggambarkan interaksi antara pengguna, halaman hasil pengecekan, sistem, dan database dalam proses pengunduhan dokumen. Proses dimulai ketika pengguna membuka hasil pengecekan dan menekan tombol “Unduh Laporan”. Halaman kemudian mengirimkan permintaan unduh kepada sistem. Sistem mengambil data hasil pengecekan dari database dan menggunakan data tersebut untuk membuat dokumen laporan. Setelah dokumen dibuat, sistem melakukan pengecekan terhadap keberhasilan proses pembuatan dokumen. Jika berhasil, sistem mengirimkan dokumen kepada halaman untuk kemudian diunduh oleh pengguna. Jika proses pembuatan dokumen gagal, sistem mengirimkan status gagal dan halaman menampilkan pesan “Dokumen gagal diunduh” kepada pengguna.

3.4.12. Sequence Merevisi Hasil Pengecekan Dokumen



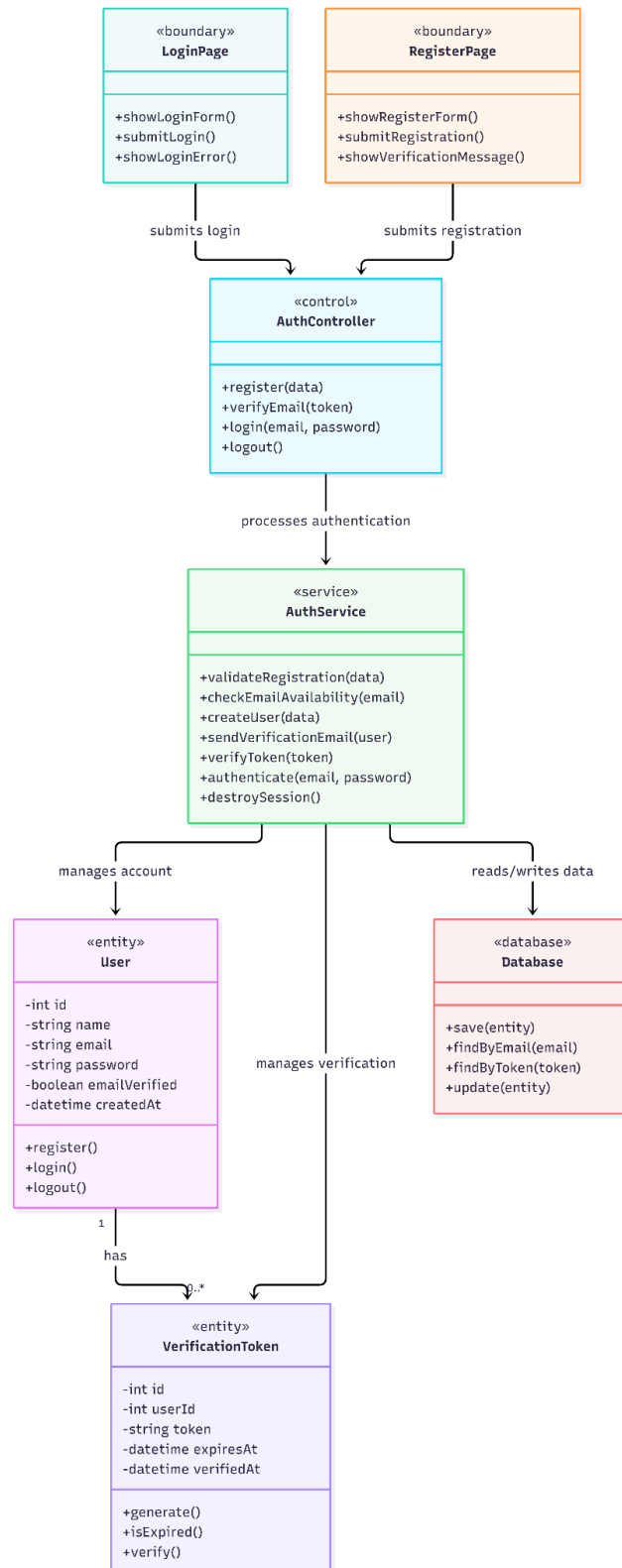
Gambar 3.4.12. Sequence Merevisi Hasil Pengecekan Dokumen

Proses revisi hasil pengecekan dokumen yang dijabarkan dalam Gambar 3.4.12 dimulai ketika pengguna membuka hasil pengecekan pada aplikasi. Aplikasi kemudian meminta *HasilPengecekanService* untuk mengambil data hasil pengecekan berdasarkan *idHasil*, lalu service mengambil data tersebut dari *Database* dan mengembalikannya ke aplikasi untuk ditampilkan kepada pengguna. Selanjutnya, pengguna membuka tab daftar pustaka, kemudian aplikasi meminta daftar pustaka berdasarkan *idHasil* kepada *HasilPengecekanService*. Data daftar pustaka diambil dari *Database* dan dikirim kembali ke aplikasi untuk ditampilkan sebagai detail referensi. Setelah itu, pengguna memilih referensi yang kategorinya keliru. Aplikasi meminta detail referensi berdasarkan *idReferensi* kepada *HasilPengecekanService*, lalu service mengambil detail referensi dari *Database* dan mengembalikannya ke aplikasi untuk ditampilkan.

Selanjutnya, pengguna mengubah kategori referensi. Aplikasi mengirimkan *idReferensi* dan kategori baru kepada *HasilPengecekanService* untuk diproses. Jika kategori baru valid, perubahan disimpan ke *Database* dan service mengembalikan informasi bahwa kategori berhasil diubah. Aplikasi kemudian menampilkan pesan "Kategori referensi berhasil diupdate" kepada pengguna. Jika kategori baru tidak valid, perubahan ditolak dan aplikasi menampilkan pesan "Terjadi kesalahan", kemudian pengguna diminta memilih kembali referensi yang kategorinya keliru. Setelah perubahan berhasil dilakukan, pengguna melihat kategori yang sudah diubah. Aplikasi kembali meminta detail referensi kepada *HasilPengecekanService*, lalu service mengambil kategori terbaru dari *Database* dan mengembalikannya ke aplikasi untuk ditampilkan kepada pengguna.

3.5 Class Diagram

3.5.1. Class Diagram Authentication

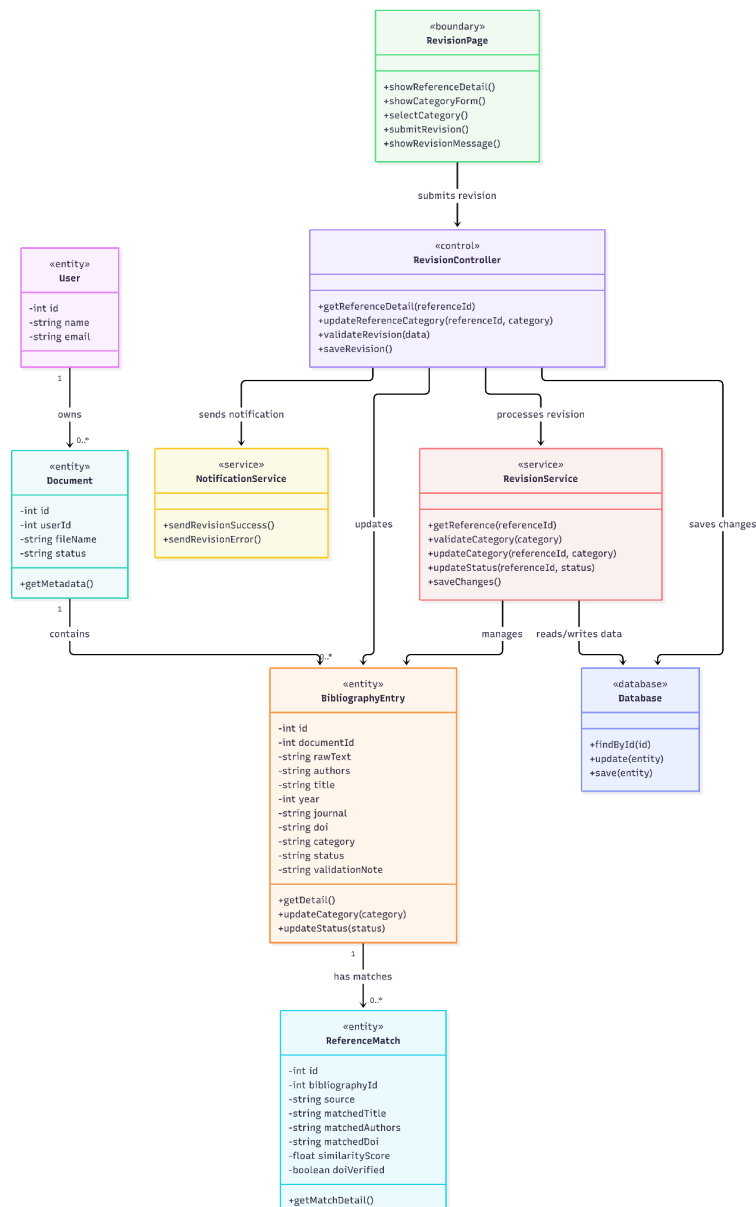


Gambar 3.5.1. Class Diagram Authentication

Class Diagram *Authentication* Menggambarkan proses pengguna melakukan *login* dan *register* ke dalam sistem. Komponen utamanya adalah User sebagai entitas yang menyimpan data pengguna, LoginPage dan RegisterPage sebagai halaman antarmuka, serta AuthController yang mengatur permintaan *autentikasi*. AuthService bertugas memproses validasi data, membuat akun, memeriksa kredensial, dan mengelola sesi pengguna.

Selain itu, terdapat VerificationToken yang digunakan untuk proses verifikasi email sebelum akun dapat digunakan. Data pengguna dan token disimpan pada Database. Alur hubungan kelas dimulai dari halaman antarmuka menuju AuthController, kemudian diteruskan ke AuthService untuk diproses dan disimpan atau diperiksa pada Database.

3.5.2. Class Diagram Upload Dokumen

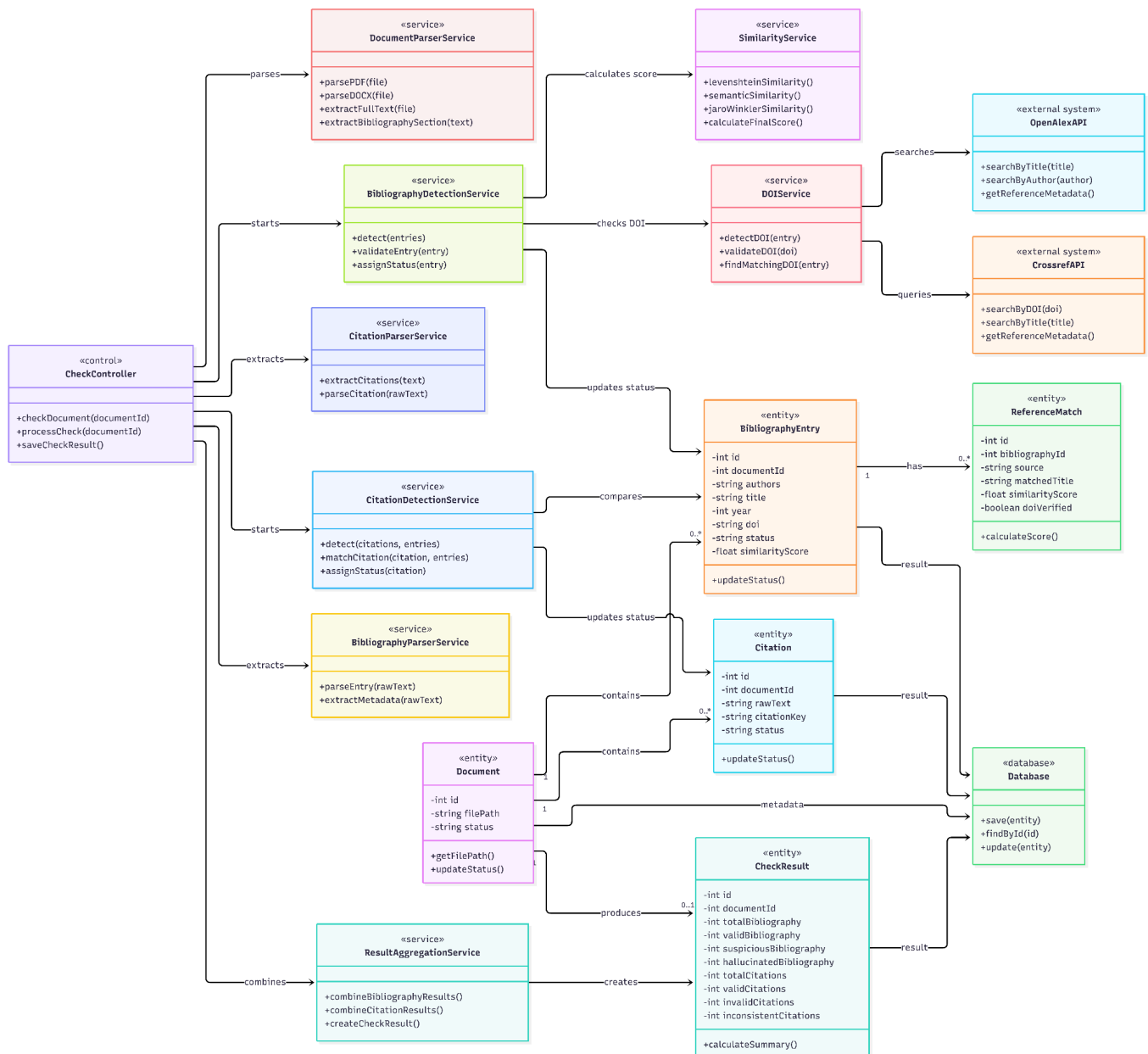


Gambar 3.5.2. Class Diagram Upload Dokumen

Class Diagram *Upload* Dokumen menjelaskan proses pengguna mengunggah dokumen ke sistem. UploadPage menjadi antarmuka untuk memilih dan mengirim dokumen, sedangkan DocumentController mengatur proses *upload*, validasi, dan penyimpanan informasi dokumen. FileValidationService memeriksa format serta ukuran file agar sesuai dengan ketentuan sistem.

Document menyimpan informasi seperti nama file, lokasi penyimpanan, format, ukuran, status, dan waktu unggah. FileStorageService bertugas menyimpan file asli ke File Storage, sedangkan *metadata* dokumen disimpan ke Database. Pemisahan ini membuat Database tidak perlu menyimpan file secara langsung, tetapi cukup menyimpan *path* atau lokasi file tersebut.

3.5.3. Class Diagram Mengecek Dokumen

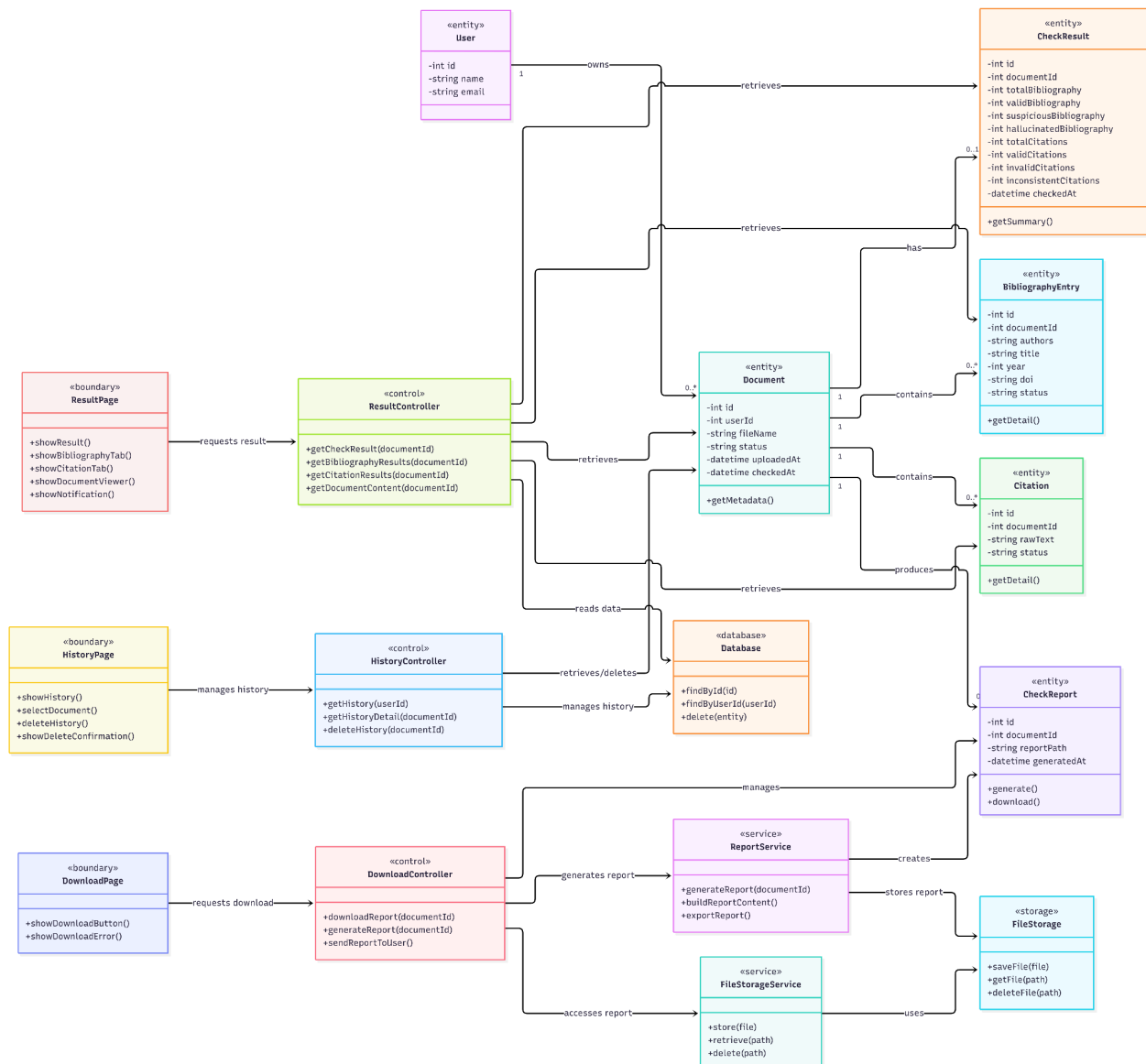


Gambar 3.5.3. Class Diagram Mengecek Dokumen

Class Diagram Mengecek Dokumen merupakan bagian utama sistem yang menggambarkan proses pemeriksaan dokumen. CheckController mengatur seluruh proses pengecekan, mulai dari mengambil dokumen, menjalankan *parsing*, mengekstraksi daftar pustaka dan sitasi, hingga menyimpan hasil pemeriksaan.

DocumentParserService, BibliographyParserService, dan CitationParserService bertugas mengambil informasi dari dokumen. BibliographyDetectionService memeriksa daftar pustaka menggunakan DOIService, SimilarityService, CrossrefAPI, dan OpenAlexAPI. Hasil pencocokan disimpan melalui ReferenceMatch. Selanjutnya, CitationDetectionService memeriksa kesesuaian sitasi dengan daftar pustaka. Seluruh hasil kemudian digabungkan oleh ResultAggregationService dan disimpan sebagai CheckResult pada Database.

3.5.4. Class Diagram Melihat Hasil, Riwayat, dan Mengunduh Laporan

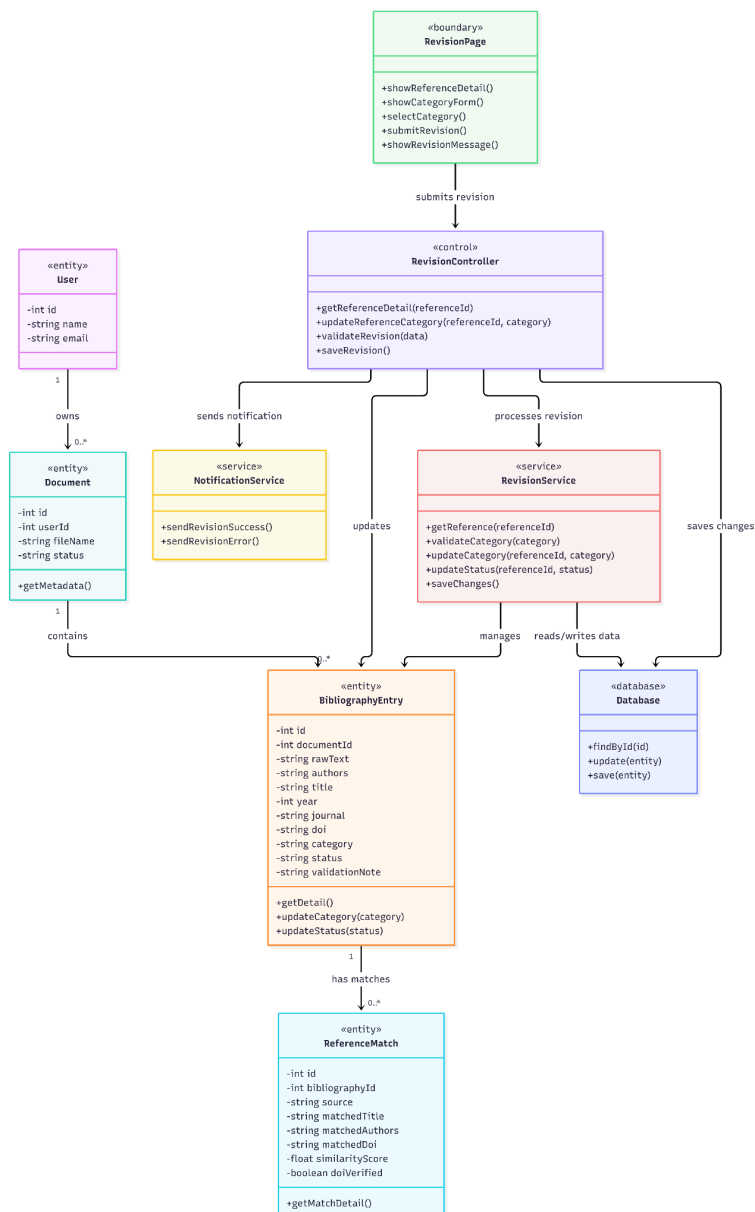


Gambar 3.5.4. *Class Diagram* Melihat Hasil, Riwayat, dan Mengunduh Laporan

Class Diagram ini menggambarkan proses pengguna melihat hasil pengecekan, melihat riwayat dokumen, serta mengunduh laporan. ResultPage menampilkan hasil pemeriksaan melalui ResultController, sedangkan HistoryPage digunakan untuk menampilkan daftar dokumen yang pernah diperiksa melalui HistoryController.

DownloadPage digunakan ketika pengguna ingin mengunduh laporan hasil pengecekan. DownloadController bekerja sama dengan ReportService untuk membentuk laporan berdasarkan CheckResult, BibliographyEntry, dan Citation. FileStorageService kemudian mengambil atau menyimpan file laporan pada File Storage. HistoryController juga dapat menghapus data riwayat tertentu dari Database sesuai permintaan pengguna.

3.5.5. Class Diagram Revisi Hasil Pengecekan

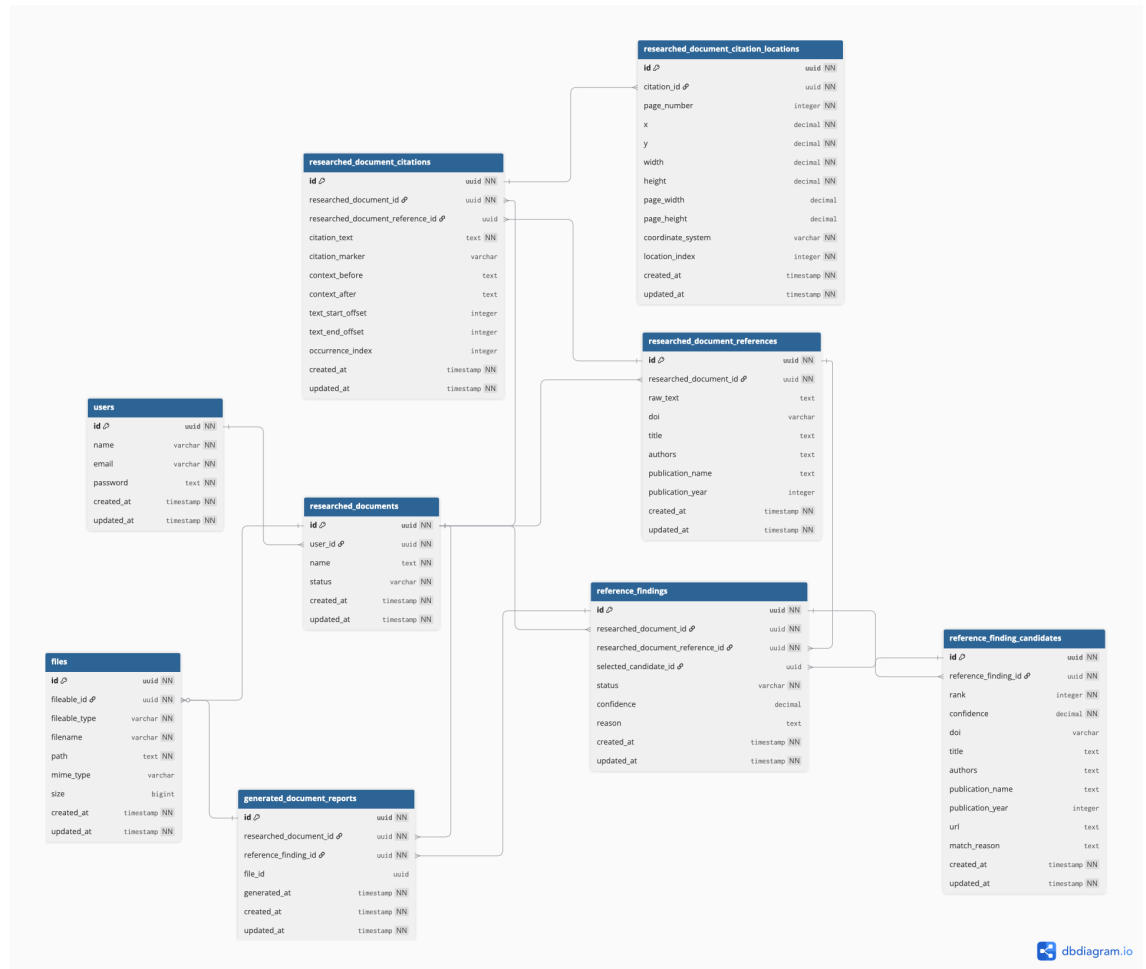


Gambar 3.5.5. Class Diagram Revisi Hasil Pengecekan

Class Diagram Revisi Hasil Pengecekan menjelaskan proses pengguna memperbarui kategori atau status hasil pemeriksaan daftar pustaka. **RevisionPage** menjadi halaman antarmuka, sedangkan **RevisionController** menerima permintaan perubahan dan meneruskannya kepada **RevisionService**. **RevisionService** melakukan validasi kategori, memperbarui status, dan menyimpan perubahan pada **BibliographyEntry** di **Database**. **ReferenceMatch** digunakan sebagai informasi pendukung mengenai hasil pencocokan referensi. Setelah perubahan berhasil disimpan, **NotificationService** dapat memberikan pemberitahuan kepada pengguna bahwa revisi telah berhasil dilakukan.

3.6 Entity Relationship Diagram

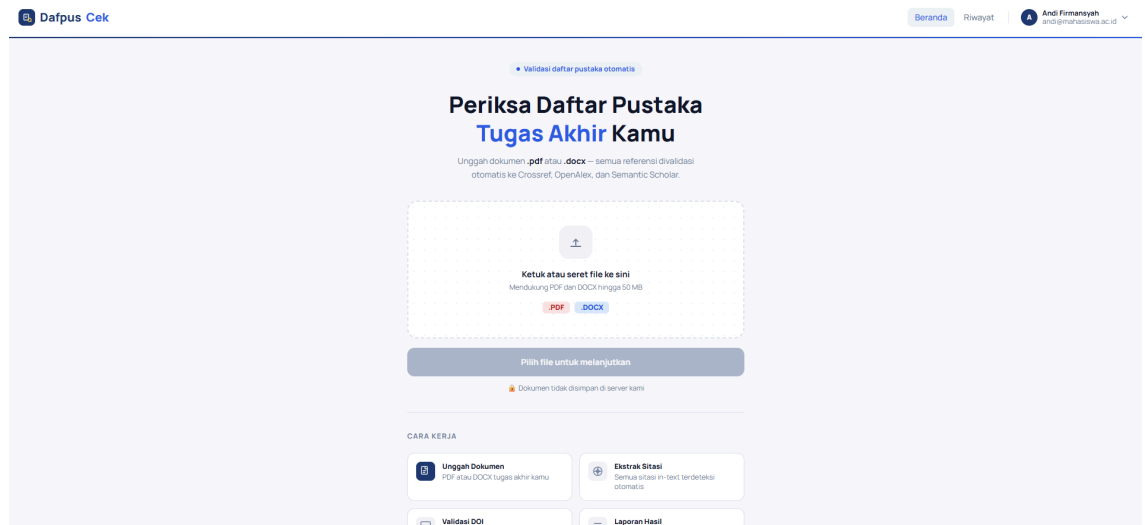
Adapun rancangan *Entity Relationship Diagram* (ERD) untuk proyek ini adalah sebagai berikut.



Dalam diagram hubungan entitas pada gambar 3.x, terdapat 9 entitas yang saling berhubungan. Secara garis besar, setiap user bisa memiliki dokumen (*researched_documents*) yang direpresentasikan / disimpan dalam sistem melalui *files*. Dokumen tersebut kemudian dapat memiliki banyak referensi (*researched_document_references*) yang setiap sitasinya dalam dokumen juga disimpan (*researched_document_citations*). Lebih lanjut, referensi yang ada dalam dokumen memiliki hasil pencarian yang digunakan sebagai kandidat hasil verifikasi referensi (*reference_findings* dan *reference_finding_candidates*). Terakhir, hasil laporan dokumen (*generated_document_reports*) berisi hasil penelusuran dan pencocokan referensi maupun sitasi berdasarkan dokumen yang diunggah.

3.7 Desain Antarmuka

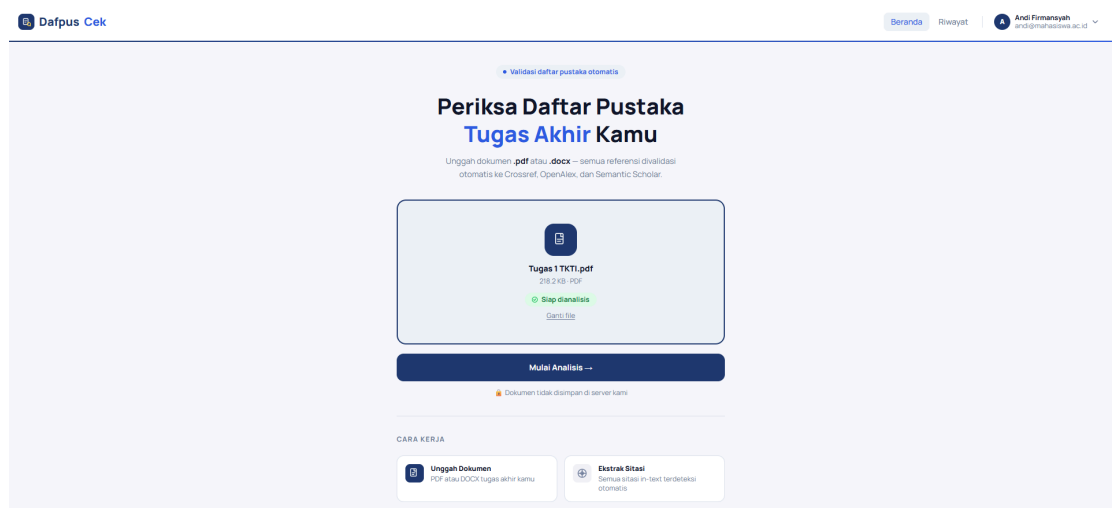
3.7.1. Desain Antarmuka Halaman Unggah Dokumen



Gambar 3.7.1. Desain Antarmuka Halaman Unggah Dokumen

Halaman unggah dokumen merupakan halaman utama yang digunakan pengguna untuk memulai proses pemeriksaan dokumen. Pada halaman ini terdapat area untuk mengunggah dokumen dalam format PDF atau DOCX serta tombol untuk melanjutkan ke proses analisis. Selain itu, halaman menyediakan navigasi menuju halaman beranda dan riwayat pemeriksaan, serta informasi singkat mengenai cara kerja sistem. Area informasi tersebut menjelaskan bahwa sistem akan mengekstraksi sitasi, melakukan validasi DOI, dan menghasilkan laporan berdasarkan hasil pemeriksaan.

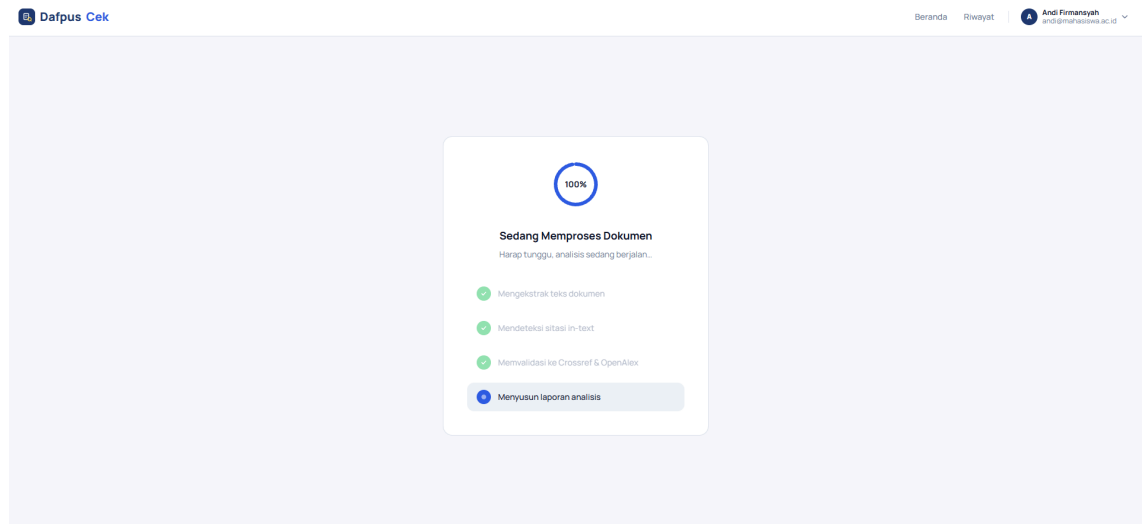
3.7.2. Desain Antarmuka Setelah Dokumen Dipilih



Gambar 3.7.2. Desain Antarmuka Setelah Dokumen Dipilih

Setelah pengguna memilih dokumen, sistem menampilkan informasi mengenai file yang telah dipilih, seperti nama file, ukuran, dan format dokumen. Area unggah berubah menjadi tampilan file yang menunjukkan bahwa dokumen telah berhasil dimuat dan siap dianalisis. Tombol “Mulai Analisis” kemudian dapat digunakan pengguna untuk memulai proses pemeriksaan. Desain ini memberikan konfirmasi kepada pengguna bahwa dokumen yang akan diperiksa telah sesuai sebelum proses analisis dijalankan.

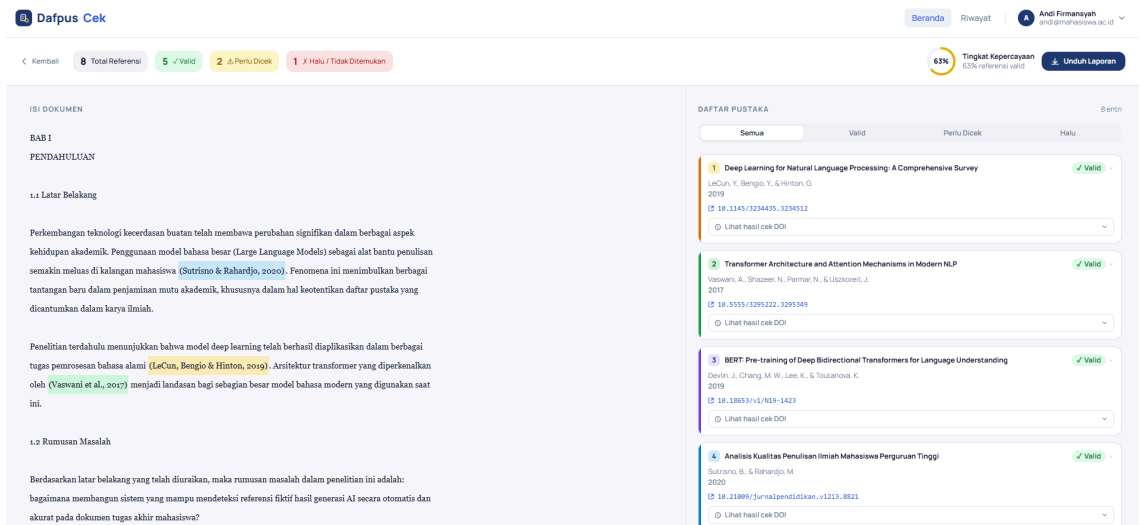
3.7.3. Desain Antarmuka Proses Analisis



Gambar 3.7.3. Desain Antarmuka Proses Analisis

Halaman proses analisis digunakan untuk memberikan informasi mengenai tahapan pemeriksaan yang sedang dilakukan oleh sistem. Pada halaman ini ditampilkan indikator persentase proses serta beberapa tahapan, seperti mengekstrak teks dokumen, mendeteksi sitasi dalam teks, melakukan validasi melalui layanan eksternal, dan menyusun laporan analisis. Setiap tahapan diberikan indikator untuk menunjukkan proses yang telah selesai maupun proses yang sedang berjalan. Tampilan ini bertujuan memberikan informasi kepada pengguna selama sistem melakukan pemrosesan sehingga pengguna mengetahui bahwa dokumen sedang dianalisis.

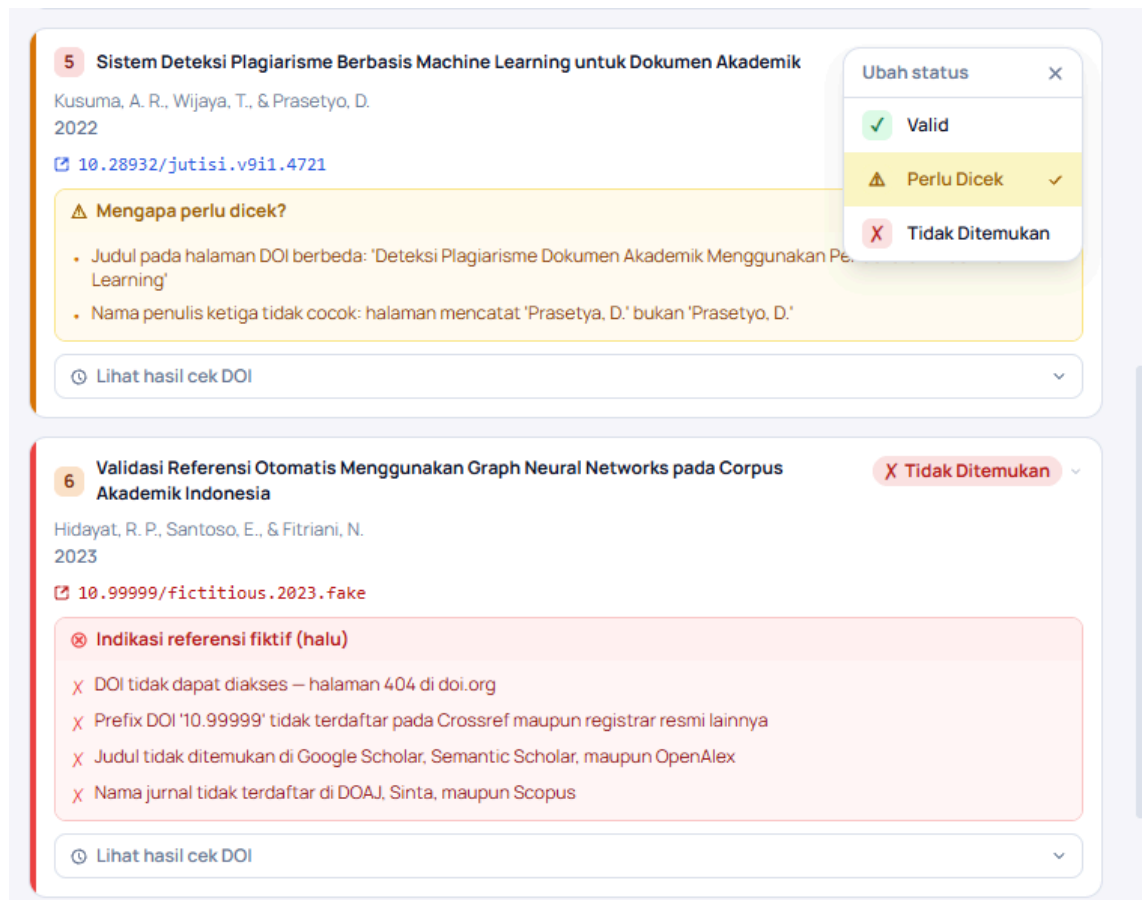
3.7.4. Desain Antarmuka Hasil Pengecekan



Gambar 3.7.4. Desain Antarmuka Hasil Pengecekan

Halaman hasil pengecekan digunakan untuk menampilkan hasil analisis daftar pustaka dan sitasi dari dokumen yang telah diperiksa. Bagian utama halaman dibagi menjadi dua area, yaitu isi dokumen pada sisi kiri dan daftar pustaka pada sisi kanan. Pada bagian daftar pustaka ditampilkan jumlah referensi beserta statusnya, seperti valid, perlu diperiksa, dan halu atau tidak ditemukan, sedangkan bagian isi dokumen digunakan untuk menunjukkan bagian yang berkaitan dengan hasil pemeriksaan. Sistem juga menampilkan tingkat kepercayaan dan ringkasan jumlah referensi berdasarkan hasil analisis sehingga pengguna dapat mengetahui kondisi dokumen secara keseluruhan.

3.7.5. Desain Antarmuka Detail Status Referensi



Gambar 3.7.5. Desain Antarmuka Detail Status Referensi

Antarmuka detail status referensi digunakan ketika pengguna ingin melihat informasi lebih lengkap mengenai hasil pemeriksaan suatu referensi. Pada tampilan ini ditunjukkan informasi referensi, nama penulis, DOI, serta status hasil pemeriksaan. Untuk referensi yang memerlukan pemeriksaan lebih lanjut, sistem memberikan alasan seperti ketidaksesuaian judul atau nama penulis, sedangkan referensi yang tidak ditemukan dapat diberikan informasi mengenai DOI atau metadata yang tidak berhasil diverifikasi. Pengguna juga dapat mengubah status referensi melalui pilihan seperti Valid, Perlu Dicek, dan Tidak Ditemukan, sehingga hasil pemeriksaan dapat ditinjau kembali secara manual.

3.7.6. Desain Antarmuka Riwayat Analisis

Riwayat Analisis
5 dokumen telah dianalisis

Cari nama file...

Semua ≥ 90% 70-89% < 70% Terbaru

DOKUMEN	REFERENSI	HALU	PERLU DICEK	KEPERCAYAAN	
Skripsi_Andi_Firmanayah_2024.pdf 7 Sep 2024, 14:50 - 2.4 MB - 87 hal	32	X 2	4	81%	
TA_Sistem_Rekomendasi_Nurul.docx 6 Sep 2024, 09:15 - 11 MB - 64 hal	18	✓ 0	—	100%	
Draft_TA_Beti-1_Ricky.pdf 4 Sep 2024, 16:48 - 3.8 MB - 102 hal	45	X 8	8	64%	
Proposal_TA_Dewi_Kusumawati.docx 2 Sep 2024, 11:40 - 10.5 MB - 29 hal	12	✓ 0	1	92%	
Revisi_Skripsi_Final_Bagas.pdf 31 Agu 2024, 20:05 - 5.1 MB - 134 hal	55	X 8	9	69%	

Menampilkan 5 dari 5 dokumen

Gambar 3.7.6. Desain Antarmuka Riwayat Analisis

Halaman riwayat analisis digunakan untuk menampilkan daftar dokumen yang sebelumnya telah diperiksa oleh pengguna. Setiap data riwayat menampilkan informasi seperti nama dokumen, jumlah referensi, jumlah referensi yang terindikasi halus, jumlah referensi yang perlu diperiksa, serta tingkat kepercayaan hasil analisis. Tersedia juga fitur pencarian dan penyaringan untuk membantu pengguna menemukan dokumen tertentu berdasarkan nama atau kategori hasil pemeriksaan. Pengguna dapat membuka kembali hasil pemeriksaan tanpa perlu melakukan analisis ulang serta dapat menghapus riwayat dokumen yang tidak diperlukan.

RENCANA PELAKSANAAN

4.1 Rencana Kegiatan

Berikut adalah rencana proyek serta timelinenya untuk *project capstone* yang akan kami kerjakan:

Tabel 4.1 Rencana Kegiatan

Kegiatan	September					Oktober					November				Desember				
	1	2	3	4	5	1	2	3	4	5	1	2	3	4	1	2	3	4	5
Analisis Kebutuhan dan Perancangan Sistem																			
Pengembangan Sistem																			
Pengembangan Model/Algoritma Deteksi																			
Integrasi dan Pengujian Sistem																			
Evaluasi dan Perbaikan Sistem																			
Expo Project Capstone																			
Penyusunan Panduan Penggunaan Aplikasi																			
Presentasi Akhir dan Pengumpulan																			

4.2 Pembagian Tugas

Berikut adalah tabel pembagian tugas untuk *project capstone* yang akan kami kerjakan:

Tabel 4.2 Pembagian Tugas

No	NIM	Nama Mahasiswa	Peran	Tanggung Jawab
1.	11231019	Daniel Belawa Koten	Project Manager	Mengatur pembagian tugas, mengoordinasikan anggota, memantau perkembangan proyek, dan memastikan proyek berjalan sesuai rencana.
2.	11231024	Fadhil Zulfikar	Frontend & UI/UX	Membuat tampilan aplikasi, merancang antarmuka, dan mengimplementasikan fitur pada bagian frontend.
3.	11231061	Muhammad Rayhan Saputra	Frontend & UI/UX	Membantu merancang tampilan aplikasi, membuat komponen antarmuka, dan mengembangkan fitur frontend.
4.	11221079	Andy Cole	Backend & AI	Mengembangkan backend, menghubungkan sistem dengan database dan API, serta membantu pengembangan algoritma deteksi.
5.	11231015	Bagus Nur Andiansyah	Backend & AI	Mengembangkan backend, mengelola proses pengolahan data, serta membantu implementasi dan pengujian algoritma AI.
6.	11231062	Muhammad Rivaldi	Backend & AI	Mengembangkan fitur backend, melakukan integrasi layanan yang dibutuhkan sistem, serta membantu pengembangan algoritma deteksi.
7.	11231033	Juan Resq Sukardi	Backend & AI	Membantu pengembangan backend, integrasi model dan algoritma, serta

				melakukan pengujian hasil deteksi.
--	--	--	--	---------------------------------------

DAFTAR PUSTAKA

- American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association. <https://doi.org/10.1037/0000165-000>
- Boretti, A. (2026). Hallucinations in generative AI: A threat to scholarly integrity and the urgent need for publisher-led academically supervised verification. *Energy Research & Social Science*, 136, 104720.
- Crossref. (2026). Register and maintain your records. Crossref.
- Dikert, K., et al. (2022). Why and how is Scrum being adapted in practice: A systematic review. *Journal of Systems and Software*, 183, 111110.
- Edison, H., Wang, X., & Conboy, K. (2022). Comparing methods for large-scale agile software development: A systematic literature review. *IEEE Transactions on Software Engineering*, 48(8), 2709–2731.
- GROBID. (2026). General principles. GROBID Documentation.
- Guder, L., Aires, J. P., da Silva, H. U., Meneguzzi, F., & Griebler, D. (2026). Sentence representations for semantic textual similarity: A systematic review. *Computer Speech & Language*, 100, 101970.
- Han, X., et al. (2026). Phantom citations: An empirical study of non-existent and unverifiable references in scholarly literature. *Journal of the Association for Information Science and Technology*.
- International DOI Foundation. (2025). DOI handbook. International DOI Foundation.
- Kokol, P., et al. (2024). Evaluation metrics and statistical tests for machine learning.
- Marx, M., et al. (2022). Reliance on science by inventors: Hybrid extraction of in-text patent-to-article citations. *Journal of Economics & Management Strategy*.
- Pikies, M., & Ali, J. (2021). Analysis and safety engineering of fuzzy string matching algorithms. *ISA Transactions*, 113, 1–8.
- Pîslaru, C., et al. (2023). GPU-based similarity metrics computation and machine learning approaches for string similarity evaluation in large datasets. *Soft Computing*.
- Ozkan, N., & Gok, M. S. (2022). Definition synthesis of agility in software development: Comprehensive review of theory to practice. *International Journal of Modern Education and Computer Science*, 14(3), 26–44.
- Trihardianingsih, L., Istighosah, M., Alin, A. Y., & Asgar, M. R. G. (2023). Systematic literature review of trend and characteristic agile model. *Jurnal Teknik Informatika*, 16(1).

Walters, W. H., & Wilder, E. I. (2023). Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13, 14045.